

Úvod do jazyka Python

Grafické uživatelské rozhraní

Vladimír Župka 2022

Obsah

Obsah	2
Použité zdroje informací	4
Grafické uživatelské rozhraní (GUI)	4
GUI Tkinter	5
Minimální program	5
Řízení událostí	6
Vytvoření databázových tabulek pro příklady	8
Dotaz zapsaný uživatelem	10
Hlavní část programu	11
Vytvoření okna a první dotaz pro zjištění jmen sloupců	11
Skladba okna a první dotaz pro zjištění dat	12
Funkce zobrazení tabulky	13
Celý program	14
Aktualizace databázové tabulky	16
Přehled činnosti programu	16
Změna řádku	18
Nový řádek	18
Smazání řádku	19
Hlavní část programu	19
Vytvoření okna dialogu pro zadání databázové tabulky	19
Vytvoření hlavního okna	21
Vytvoření dialogu pro změnu záznamů	23
Funkce tlačítek	26
Tlačítka dialogu zadání databáze	26
Tlačítka hlavního okna	28
Celý program	31
GUI PySimpleGUI	37
Minimální program	37
Řízení událostí	38
Konstrukce okna s klíči	38
Řídicí cyklus s vyhodnocením klíčů	39
Vysvětlivky	39
Dotaz zapsaný uživatelem	40
Hlavní část programu	41
Příprava a provedení dotazu poprvé	41
Konstrukce obrazové tabulky a okna	41
Zpracování událostí v cyklu	42
Funkce data_tabulky()	42
Celý program	43
Aktualizace databázové tabulky	44
Dialogové okno pro zadání schématu a tabulky	44
Konstrukce hlavního okna s obrazovou tabulkou	46
Zpracování událostí v hlavním okně	47

Vnější cyklus	47
Vnitřní cyklus	50
Zobrazení tabulky	52
Celý program	53
Distribuce programu jako aplikace.....	58
Pro Windows a macOS	58
Windows.....	58
macOS	58
Jen pro macOS	59

Použité zdroje informací

Dokumentace k *Tkinter* <https://docs.python.org/3/library/tkinter.html>.

Dokumentace k *PySimpleGUI* <https://pysimplegui.readthedocs.io/en/latest/>.

Příklady k *PySimpleGUI* <https://github.com/PySimpleGUI/PySimpleGUI/tree/master/DemoPrograms>

Dokumentace ke zjišťování typů sloupců databázové tabulky: *sqlite-utils 3.6*, kapitola *Python API Introspection* <https://sqlite-utils.datasette.io/en/3.6/index.html>

Grafické uživatelské rozhraní (GUI)

Grafické uživatelské rozhraní, anglicky Graphical User Interface (GUI) budeme chápat jako zobrazování informací na obrazovce počítače formou oken. Okna obsahují grafické a písemné prvky jako tlačítka, textová pole, obrázky, atd. Prvky se ovládají v programu na základě signálů vytvořených počítačovou myší nebo klávesnicí, ale i dotykem prstů apod.

Pro jazyk Python existuje řada programových balíků řešících uživatelské rozhraní. Jejich seznam najdeme v dokumentaci na stránce <https://wiki.python.org/moin/GuiProgramming>.

Soustavy GUI interně používají třídy a jejich dědičnost. Také v aplikacích se používá dědičnost (většinou jednoduchá), ale v příkladech ji nebudeme potřebovat, nahradíme ji použitím *funkcí*.

K ovládání činnosti prvků (oken, tlačítek apod.) slouží koncept *událostí* (*events*). vzniklých na základě výše zmíněných signálů. Většina soustav soustřeďuje kontrolu událostí uvnitř základní třídy ve formě cyklu nazývaného *mainloop* (*Tkinter*), *run* (*Kivy*), *sys.exit* (*PyQt*), *MainLoop* (*wxPython*) apod. Z nich se vymyká balík *PySimpleGUI*, který nechává cyklickou kontrolu událostí k dispozici uživateli.

Programy, které zde vytvoříme v prostředí IDE *PyCharm*, můžeme také v tomto prostředí ladit a spouštět. Ukážeme také, jak se z takových programů dá vytvořit *aplikace*, která se spouští poklepnutím (*double-click*) na *ikonu*, a to jak v systému *macOS*, tak v systému *Windows*.

Pro tuto část kurzu byly vybrány soustavy – *Tkinter* a *PySimpleGUI*.

GUI Tkinter

Balík *Tkinter* se nemusí instalovat, je součástí standardních modulů Pythonu. Hlavní modul se jmenuje `tkinter`. K němu patří ještě jeho podmodul `ttk`, který byl v průběhu vývoje balíku přidán jako jeho modernizace. Tím se poněkud znehlednila jak dokumentace, tak způsob použití tříd a funkcí, protože některé lze použít ve starším provedení (`tkinter`) i v novém provedení (`ttk`).

Importuje se buď jen jeden modul (`tkinter`) nebo i druhý modul (`ttk`) nejčastěji dvěma způsoby:

```
from tkinter import *
from tkinter import ttk

nebo

from tkinter import *
import tkinter.ttk as ttk
```

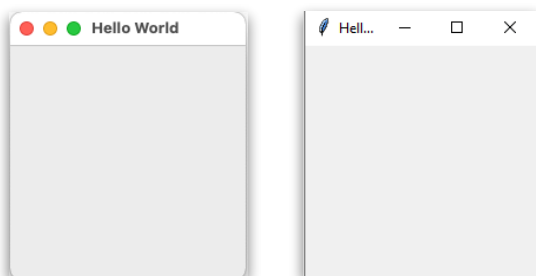
Minimální program

Minimální program Hello World vypadá takto:

```
from tkinter import *

okno = Tk()          # vytvoření hlavního okna
okno.title('Hello World') # přidání titulu do záhlaví
okno.mainloop()     # zahájení řídicího cyklu událostí (events)
```

Program zobrazí prázdné okno s titulkem, který jsme zadali. Ovládací prvky odpovídají platformě, na které program běží. Rozměry a umístění okna jsou automatické (macOS a Windows).



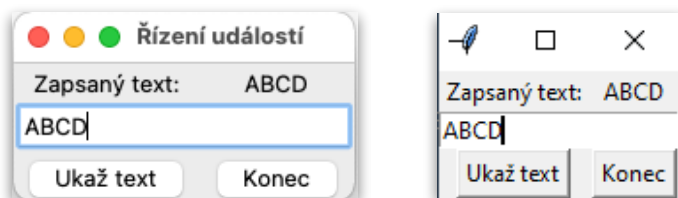
- `Tk` je hlavní třída modulu `tkinter` a volá se bez argumentu,
- `mainloop` a `title` jsou metody třídy `tkinter`.

Budeme používat hlavně prvky vytvářené z následujících tříd:

- `ttk.Label` – statický text (nápis),
- `ttk.Entry` – vstupní textové pole,
- `ttk.Button` – tlačítko,
- `ttk.Frame` – rámeček pro ukládání dalších prvků,
- `ttk.Treeview` – stromová struktura s prvky umožňujícími sloupcovou strukturu (tabulku),
- `ttk.Scrollbar` – posuvník řádků,
- `tkinter.Text` – textová oblast.

Řízení událostí

Program `ridici_cyklus_tk.py` zobrazí okno s nápisem, vstupním polem a dvěma tlačítky. Když uživatel zapíše text do vstupního pole a stiskne tlačítko "Ukaž text:", zobrazí se tento text za nápisem. Když uživatel stiskne tlačítko "Konec" nebo klepne na uzávěr okna, okno zmizí a program skončí.



Okno je rozvrženo do tří částí ve svislém směru (řádků) a dvou částí ve vodorovném směru (sloupců):

column=0	column=1	
Zapsaný text:		row=0
ABCD		row=1
Ukaž text	Konec	row=2

Takovéto uspořádání se nazývá **grid** (mřížka) a odpovídá mu stejně nazvaná metoda `grid()`, která podle parametrů `column` a `row` rozmísťuje jednotlivé prvky. Existují ještě další způsoby rozmístění prvků, ale nebudeme je používat.

Činnost programu je řízena metodou `mainloop()`, která čeká na události. Nastane-li událost stiskem tlačítka, provede funkci uvedenou v jeho definici parametrem **command**. Takové funkci se říká *handler* nebo *callback*.

Následuje komentovaný text programu.

```
from tkinter import *
from tkinter import ttk

okno = Tk()
okno.title('Řízení událostí')
```

```
def ukaz_text():
    vystup.set(vstup.get()) 1

def konec():
    okno.destroy() 2
```

- 1 Ve funkci tlačítka "Ukaž text" nejdříve metoda `get()` přečte hodnotu z proměnné `vstup` a tuto hodnotu pak metoda `set()` запиše do proměnné `vystup` (a do výstupního textu).
- 2 Ve funkci tlačítka "Konec" je metodou `destroy()` zničen objekt okna a program končí.

```
ttk.Label(okno, text='Zapsaný text:').grid(column=0, row=0) 1
vystup = StringVar() 2
ttk.Label(okno, textvariable=vystup).grid(column=1, row=0) 3
vstup = StringVar()
ttk.Entry(okno, width=20, textvariable=vstup).grid(
    column=0, row=1, columnspan=2) 4
ttk.Button(okno, text='Ukaž text', command=ukaz_text).grid(column=0, row=2) 5
ttk.Button(okno, text='Konec', command=konec).grid(column=1, row=2)

okno.mainloop() 6
```

- 1 Objekt třídy `ttk.Label` představuje výstupní text "Zapsaný text:" umístěný v mřížce do sloupce 0, řádku 0.
- 2 Proměnná `vystup` je objekt třídy `StringVar`, v němž je uložena hodnota před zápisem do výstupního prvku (Label).
- 3 Výstupní text třídy `ttk.Label` podle parametru `textvariable` přijímá hodnotu z proměnné `vystup` a je umístěn ve sloupci 1, řádku 0.
- 4 Vstupní textové pole typu `ttk.Entry` předává hodnotu zapsanou uživatelem do proměnné `vstup`, je umístěno ve sloupci 0, řádku 1 a podle parametru `columnspan` zabírá dva sloupce, tedy 0 a 1.
- 5 Tlačítko třídy `ttk.Button` má nápis "Ukaž text" a v parametru `command` jméno funkce `ukaz_text`, která se provede v rámci řídicího cyklu, jakmile uživatel na tlačítko klepne myší.
- 6 Na konci programu musí být zapsáno volání metody `mainloop()`.

Pravidlo: Každý prvek umístěný uvnitř nějakého jiného prvku, musí v prvním argumentu volání třídy uvést tento nadřazený prvek (zde prvek `okno`).

Vytvoření databázových tabulek pro příklady

Budeme mít k dispozici databázi SQLite3 se schematem `zasoby.db` a tabulkami `ceny` a `stavy`. Vytvoříme je programem `db_create.py`, kde u obou tabulek definujeme sloupec `id` typu `INTEGER` jako *primární klíč* s automatickým zvyšováním čísla, abychom při budoucím přepisování nebo mazání záznamu (řádku) měli na něj jednoznačný odkaz.

Při vkládání záznamů příkazem `INSERT` se jméno ani hodnota sloupce `id` nezapisuje.

Hodnotu sloupce `id` nelze v záznamu přepisovat.

```
import sqlite3

connection = sqlite3.connect('zasoby.db')

cursor = connection.cursor()

cursor.execute('DROP TABLE IF EXISTS stavy')
cursor.execute('DROP TABLE IF EXISTS ceny')

cursor.execute("""CREATE TABLE stavy (id INTEGER PRIMARY KEY AUTOINCREMENT,
závod TEXT, sklad TEXT, materiál TEXT, množství REAL)
""")

cursor.execute("""CREATE TABLE ceny (id INTEGER PRIMARY KEY AUTOINCREMENT,
materiál TEXT, cenaj REAL, název TEXT)
""")
```

```

cursor.executemany("INSERT INTO stavy (závod, sklad, materiál, množství) VALUES
(?, ?, ?, ?)",
[
    ('01', '01', '00009', 1.05),
    ('01', '02', '00002', 2.05),
    ('02', '01', '00003', 3.05),
    ('02', '02', '00001', 400.05),
    ('03', '01', '00005', 5.55),
    ('03', '02', '00002', 6.05),
    ('03', '03', '00005', 5.55),
    ('03', '04', '00002', 6.05),
    ('02', '01', '00007', 7.05),
    ('02', '02', '00008', 800.05),
    ('01', '01', '00009', 9),
    ('01', '02', '00010', 40.05),
    ('02', '01', '00001', 50.06),
    ('02', '02', '00012', 60.06),
    ('01', '01', '00013', 13),
    ('01', '02', '00012', 100.05),
    ('02', '01', '00015', 15.05),
    ('02', '02', '99999', 800.05),
])

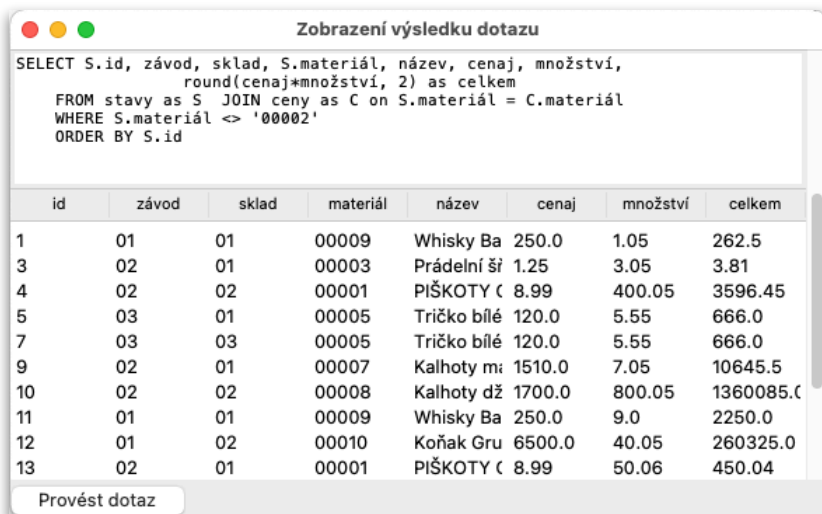
cursor.executemany("INSERT INTO ceny (materiál, cenaj, název) VALUES (?, ?, ?)",
[
    ('00001', 8.99, 'PIŠKOTY OPAVIA'),
    ('00002', 459.00, 'Zubní pasta Kalodont'),
    ('00003', 1.25, 'Prádelní šňůra'),
    ('00004', 10.50, 'Ponožky pánské tmavé'),
    ('00005', 120.00, 'Tričko bílé'),
    ('00006', 10.55, 'Ponožky pánské bílé, nové'),
    ('00007', 1510.00, 'Kalhoty manšestrové'),
    ('00008', 1700.00, 'Kalhoty džínové'),
    ('00009', 250.00, 'Whisky Balantine'),
    ('00010', 6500.00, 'Koňak Gruzínský'),
    ('00011', 159.00, 'Taška sportovní'),
    ('00012', 45.00, 'Pálka pingpongová'),
    ('00013', 360.00, 'Míč'),
    ('00014', 3500.00, 'Sako tvídové, nadměr'),
    ('00015', 12500.00, 'Motorové koště'),
    ('00016', 380.00, 'Slepičí peří na polštáře'),
    ('00017', 130.00, 'Hrábě dřevěné, kolíky plastové'),
    ('00018', 56.00, 'Husí sádlo v konzervě'),
    ('00452', 874.00, 'Tiskárna barevná laserová'),
    ('99999', 9999.99, 'AAAA+%BBB%_CCC ')
])

connection.commit()
cursor.close()
connection.close()

```

Dotaz zapsaný uživatelem

Program `tkdb_dotaz.py` na začátku запиše do textové oblasti vzorový dotaz (příkaz SELECT) a jeho výsledek zobrazí v obrazové tabulce.



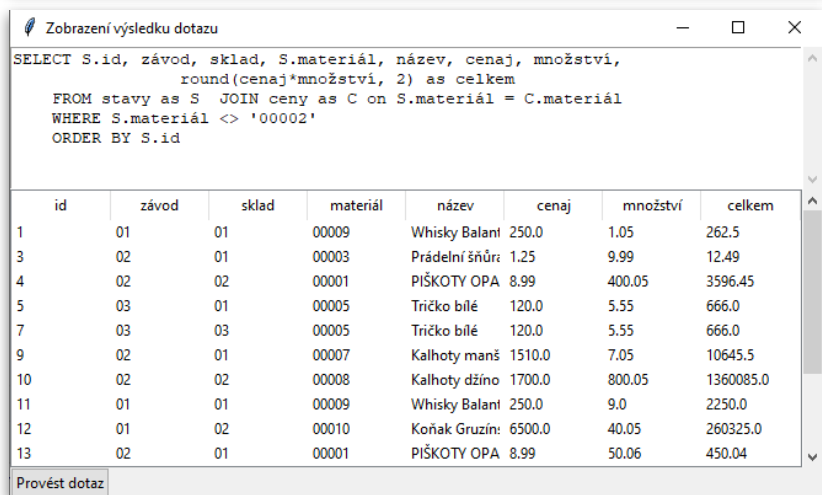
The screenshot shows a window titled "Zobrazení výsledku dotazu". It contains a text area with the following SQL query:

```
SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
       round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id
```

Below the query is a table with 8 columns: id, závod, sklad, materiál, název, cenaj, množství, celkem. It contains 13 rows of data.

id	závod	sklad	materiál	název	cenaj	množství	celkem
1	01	01	00009	Whisky Ba	250.0	1.05	262.5
3	02	01	00003	Prádelní ší	1.25	3.05	3.81
4	02	02	00001	PIŠKOTY C	8.99	400.05	3596.45
5	03	01	00005	Tričko bílé	120.0	5.55	666.0
7	03	03	00005	Tričko bílé	120.0	5.55	666.0
9	02	01	00007	Kalhoty m	1510.0	7.05	10645.5
10	02	02	00008	Kalhoty dž	1700.0	800.05	1360085.0
11	01	01	00009	Whisky Ba	250.0	9.0	2250.0
12	01	02	00010	Koňak Gru	6500.0	40.05	260325.0
13	02	01	00001	PIŠKOTY C	8.99	50.06	450.04

At the bottom of the window is a button labeled "Provést dotaz".



The screenshot shows a window titled "Zobrazení výsledku dotazu". It contains the same SQL query as the previous screenshot:

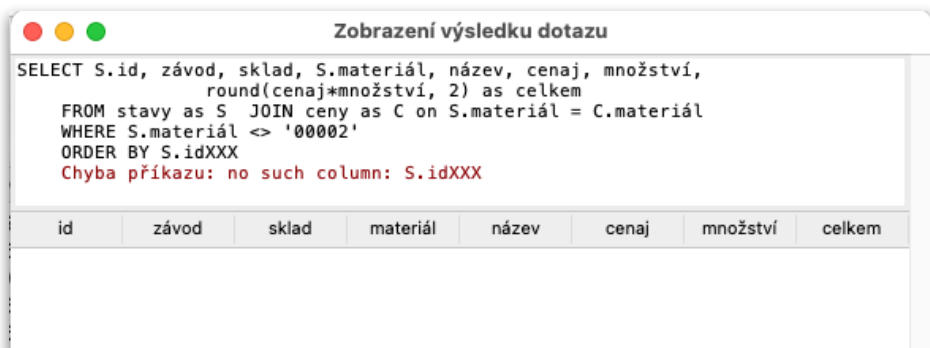
```
SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
       round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id
```

Below the query is a table with 8 columns: id, závod, sklad, materiál, název, cenaj, množství, celkem. It contains 13 rows of data.

id	závod	sklad	materiál	název	cenaj	množství	celkem
1	01	01	00009	Whisky Balant	250.0	1.05	262.5
3	02	01	00003	Prádelní šňůr	1.25	9.99	12.49
4	02	02	00001	PIŠKOTY OPA	8.99	400.05	3596.45
5	03	01	00005	Tričko bílé	120.0	5.55	666.0
7	03	03	00005	Tričko bílé	120.0	5.55	666.0
9	02	01	00007	Kalhoty manš	1510.0	7.05	10645.5
10	02	02	00008	Kalhoty džino	1700.0	800.05	1360085.0
11	01	01	00009	Whisky Balant	250.0	9.0	2250.0
12	01	02	00010	Koňak Gruzín	6500.0	40.05	260325.0
13	02	01	00001	PIŠKOTY OPA	8.99	50.06	450.04

At the bottom of the window is a button labeled "Provést dotaz".

Uživatel může text dotazu změnit nebo napsat jiný a pak stisknout tlačítko "Provést dotaz". Program pak zobrazí výsledek nového dotazu. Zapiše-li uživatel dotaz s chybou, ohlásí se chyba červeně v textové oblasti pod příkazem.



The screenshot shows a window titled "Zobrazení výsledku dotazu". It contains the following SQL query:

```
SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
       round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.idXXX
```

Below the query, a red error message is displayed: "Chyba příkazu: no such column: S.idXXX". Below the error message is an empty table with 8 columns: id, závod, sklad, materiál, název, cenaj, množství, celkem.

id	závod	sklad	materiál	název	cenaj	množství	celkem
----	-------	-------	----------	-------	-------	----------	--------

Hlavní část programu

Vytvoření okna a první dotaz pro zjištění jmen sloupců

```
okno = Tk() # vytvořím hlavní okno
okno.title('Zobrazení výsledku dotazu')
# velikost WxH je standardní(chybí), pozici +X+Y zadáváme
okno.geometry('+600+300') 1

connection = sqlite3.connect('zasoby.db') # připojím databázi
connection.row_factory = sqlite3.Row 2
cursor = connection.cursor() # definuji kurzor

# text vzorového SQL dotazu
text_dotazu = '''SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
                    round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id
'''

cursor.execute(text_dotazu) # provedu dotaz
zaznamy = cursor.fetchall() # přečtu výslednou tabulku
jmena_sl = zaznamy[0].keys() # zjistím jména sloupců z prvního záznamu 3
```

1 Velikost a pozice okna se zadává v metodě `geometry()` *textovým* schematem `WxH+X+Y`, kde `W` a `H` je šířka a výška, `X` a `Y` jsou souřadnice od levé a horní hrany obrazovky. Místo znaménka `+` může být znaménko `-`, které znamená vzdálenost od opačných konců obrazovky.

2 Po připojení databáze vytvoříme objekt `row_factory` ze třídy `sqlite.Row`, který umožní z výsledku dotazu zjistit jména sloupců výsledné tabulky (více dále ve funkci `zobrazit_dotaz()`).

3 Objekt `zaznamy[0]` (první řádek výsledné tabulky) je instance třídy `Row`. Z něj se získají jména sloupců metodou `keys()`, která použijeme pro určení hlaviček sloupců.

Skladba okna a první dotaz pro zjištění dat

```
textova_oblast = Text(okno, height=7) # oblast textu pro 7 řádků
textova_oblast.grid(column=0, row=0) 1
textova_oblast.insert('end', text_dotazu) 2

scroll_txt = ttk.Scrollbar(okno, orient=VERTICAL) 3
textova_oblast.configure(yscrollcommand=scroll_txt.set) # posuvník 4
scroll_txt.grid(column=1, row=0, sticky='NS') # roztažen shora dolů 5

obraz_tabulka = ttk.Treeview(okno, columns=jmena_sl, show='headings',
                             height=10) 6
obraz_tabulka.grid(column=0, row=1, sticky='WE') # roztažena zleva doprava
scroll_obr = ttk.Scrollbar(okno, orient=VERTICAL)
obraz_tabulka.configure(yscrollcommand=scroll_obr.set) # posuvník
scroll_obr.grid(column=1, row=1, sticky='NS') # roztažen shora dolů

tlac_zobrazit = ttk.Button(okno, text='Provést dotaz',
                           command=zobrazit_dotaz) 7
tlac_zobrazit.grid(sticky='W')

zobrazit_dotaz() # provedu dotaz poprvé

okno.mainloop()
```

- 1 Text je třída, která dovoluje zobrazovat a upravovat text v řádcích. Volání této třídy (a dalších) musí obsahovat objekt nadřazeného prvku (okno) jako první argument. Metoda `grid()` umísťuje prvek `textova_oblast` do sloupce 0 a řádku 0 pomyslné mřížky nadřazeného prvku `okno`.
- 2 Metoda `insert()` vloží text dotazu do textové oblasti. Argument `'end'` říká, že text bude umístěn za konec dosavadního textu.
- 3 `Scrollbar` je posuvník dovolující posouvat oblast daného prvku tak, aby potřebná část byla viditelná. Argument `orient=VERTICAL` určuje jeho svislou orientaci.
- 4 Metoda `configure()` s argumentem `yscrollcommand=scroll_txt.set` připojuje posuvník k textové oblasti (metodou `set()`).
- 5 Obecně se zarovnání vyjadřuje v parametru `sticky` jedním až čtyřmi písmeny W, E, N, S nebo w, e, n, s v libovolném pořadí a uzavřenými v apostrofech. Jedno písmeno se nemusí uzavírat do apostrofů. Tak například `'NS'` znamená, že prvek se rozprostře shora dolů, `'WE'` zleva doprava, `'WENS'` do obou směrů. `'W'`, `'w'`, `W` nebo `w` znamená zarovnání vlevo apod.
- 6 `Treeview` je třída, která obsahuje jak stromovou, tak tabulkovou zobrazovací strukturu. Argument `columns` určuje jména sloupců zobrazovaných v tabulce, argument `show` říká, že v tabulce budou zobrazeny hlavičky sloupců.
- 7 `Button` je třída pro tlačítko. Parametr `text` určuje nápis na tlačítku, parametr `command` určuje funkci `zobrazit_dotaz` (viz dále), která se provede při stisku tlačítka.

Funkce zobrazení tabulky

```
def zobrazit_dotaz():
    text_dotazu = textova_oblast.get('1.0', 'end') # přečtu nový dotaz 1
    obraz_tabulka.delete(* obraz_tabulka.get_children()) 2
    try:
        cursor.execute(text_dotazu) # provedu dotaz
        zaznamy = cursor.fetchall() # přečtu výslednou tabulku
        if zaznamy:
            jmena_sl = zaznamy[0].keys() # zjistím jména sloupců
            for zaznam in zaznamy: # zapíšu záznamy do obrazové tabulky
                obraz_tabulka.insert('', 'end', values=list(zaznam)) 3
            for col in range(len(jmena_sl)): # dosadím hlavičky
                obraz_tabulka.heading(col, text=jmena_sl[col]) 4
            for col in range(len(jmena_sl)): # šířka sloupců a zarovnání
                obraz_tabulka.column(col, width=60, anchor=W) 5
    except sqlite3.Error as err:
        obraz_tabulka.delete(* obraz_tabulka.get_children()) # smažu obsah
        textova_oblast.tag_configure('error', foreground='red4') 6
        textova_oblast.insert('end', f"Chyba příkazu: {err}"+'\n', 'error')
```

1 Nový obsah textové oblasti se přečte metodou `get()` od prvního znaku nultého řádku `'1.0'` do konce textu `'end'`. Pak se znovu provede dotaz a přečtou se záznamy z výsledné tabulky.

2 Metoda `delete()` smaže obsah obrazové tabulky (aby se nepřidávalo za konec). Hvězdička znamená vícenásobný (variadic) argument, který představuje všechny podřízené objekty (children) obrazové tabulky, tedy 15 řádků.

3 Metoda `insert()` zapíše v cyklu hodnoty záznamů do sloupců a řádků obrazové tabulky. Před každým zápisem musíme z objektu `zaznam` vytvořit seznam (`list`). První dva argumenty znamenají, že záznam se do řádku zapíše od začátku `' '` do konce `'end'`.

4 Metoda `heading()` zapíše v cyklu jména sloupců do hlaviček obrazové tabulky.

5 Metoda `column()` určí v cyklu šířku 60 bodů u všech sloupců a zarovnání textu vlevo.

6 Metoda `tag_configure()` určí barvu textu pro následující příkaz. Řetězec `'error'` je tzv. *tag* či značka, podle které se řídí jiný příkaz, který ji také obsahuje. Zde to je příkaz `insert()` pro zápis chyby.

Poznámka: Tato definice funkce musí být ovšem umístěna na začátku, protože se další program na ni odvolává.

Celý program

```
from tkinter import *
from tkinter import ttk
import sqlite3

# funkce zobrazení databázové tabulky v obrazové tabulce
# -----
def zobrazit_dotaz():
    text_dotazu = textova_oblast.get('1.0', 'end') # přečtu nový dotaz
    obraz_tabulka.delete(*obraz_tabulka.get_children()) # smažu obsah tabulky
    try:
        cursor.execute(text_dotazu) # provedu dotaz
        zaznamy = cursor.fetchall() # přečtu výslednou tabulku
        if zaznamy:
            jmena_sl = zaznamy[0].keys() # jména sloupců z prvního záznamu
            for zaznam in zaznamy: # zapíšu záznamy do obrazové tabulky
                obraz_tabulka.insert('', 'end', values=list(zaznam))
            for col in range(len(jmena_sl)): # dosadím hlavičky
                obraz_tabulka.heading(col, text=jmena_sl[col])
            for col in range(len(jmena_sl)): # šířka sloupců a zarovnání
                obraz_tabulka.column(col, width=60, anchor=W)

        except sqlite3.Error as err:
            obraz_tabulka.delete(* obraz_tabulka.get_children()) # smažu obsah
            textova_oblast.tag_configure('error', foreground='red4') # změna barvy
            textova_oblast.insert('end', f"Chyba příkazu: {err}" + '\n', 'error')
    # -----

# konstrukce okna
# -----
okno = Tk() # vytvořím hlavní okno
okno.title('Zobrazení výsledku dotazu')
# velikost WxH je standardní(chybí), pozici +X+Y zadáváme
okno.geometry('+600+300')

connection = sqlite3.connect('/Users/vzupka/PycharmProjects/tkinter/zasoby.db')
# vytvořím objekt ke zjišťování údajů o řádce tabulky
connection.row_factory = sqlite3.Row
cursor = connection.cursor() # definuji kurzor

# text vzorového SQL dotazu
text_dotazu = '''SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
                    round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id
'''

cursor.execute(text_dotazu) # provedu dotaz
zaznamy = cursor.fetchall() # přečtu výslednou tabulku
jmena_sl = zaznamy[0].keys() # zjistím jména sloupců z prvního záznamu

textova_oblast = Text(okno, height=7) # oblast textu pro 7 řádků
textova_oblast.grid(column=0, row=0)
textova_oblast.insert('end', text_dotazu)

scroll_txt = ttk.Scrollbar(okno, orient=VERTICAL)
textova_oblast.configure(yscrollcommand=scroll_txt.set) # posuvník
#textova_oblast['yscrollcommand'] = scroll_txt.set # alternativní způsob
```

```

scroll_txt.grid(column=1, row=0, sticky='NS') # roztažen shora dolů

obraz_tabulka = ttk.Treeview(okno, columns=jmena_sl, show='headings',
                             height=10)
obraz_tabulka.grid(column=0, row=1, sticky='WE') # roztažena zleva doprava
scroll_obr = ttk.Scrollbar(okno, orient=VERTICAL)
obraz_tabulka.configure(yscrollcommand=scroll_obr.set) # posuvník
scroll_obr.grid(column=1, row=1, sticky='NS') # roztažen shora dolů

# tlačítko k provedení dotazu
tlac_zobrazit = ttk.Button(okno, text='Provést dotaz', command=zobrazit_dotaz)
tlac_zobrazit.grid(sticky='W')

zobrazit_dotaz() # provedu dotaz poprvé

okno.mainloop()

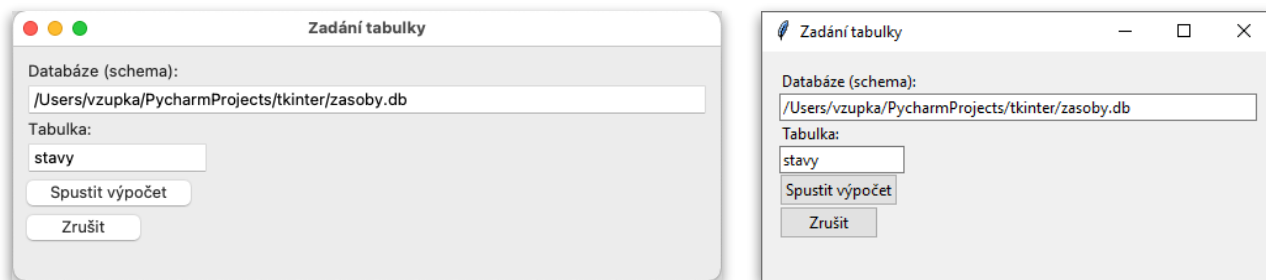
```

Aktualizace databázové tabulky

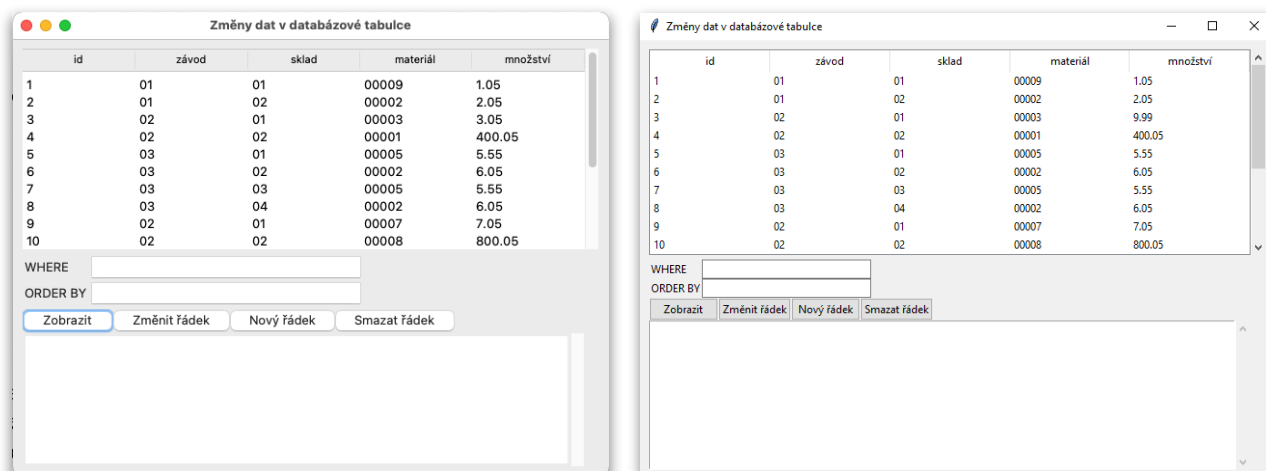
Program `tkdb_update.py` provádí kompletní údržbu databázové tabulky pomocí SQL příkazů: výběru záznamů `SELECT`, přepisu záznamu `UPDATE`, vložení záznamu `INSERT` a smazání záznamu `DELETE`. Ve srovnání z předchozím příkladem je tento příklad podstatně složitější.

Přehled činnosti programu

Nejprve zobrazí výzvu k zadání schematu a tabulky a spuštění výpočtu nebo jeho zrušení.



Po stisku tlačítka *Spustit výpočet* se zobrazí hlavní okno s obsahem tabulky a prázdným oknem protokolu. Do okna protokolu se zapisují oznámení, chyby a důležité příkazy.



Uživatel může do textových polí zapsat podmínky výběru WHERE, např.

id <=5 AND množství > 2 a seřazení ORDER BY, např. množství desc pro příkaz SELECT. Po stisku tlačítka *Zobrazit* se zobrazí vybraná tabulka ve správném seřazení řádků.

id	závod	sklad	materiál	množství
4	02	02	00001	400.05
5	03	01	00005	5.55
3	02	01	00003	3.05
2	01	02	00002	2.05

WHERE

ORDER BY

id	závod	sklad	materiál	množství
4	02	02	00001	400.05
5	03	01	00005	5.55
3	02	01	00003	3.05
2	01	02	00002	2.05

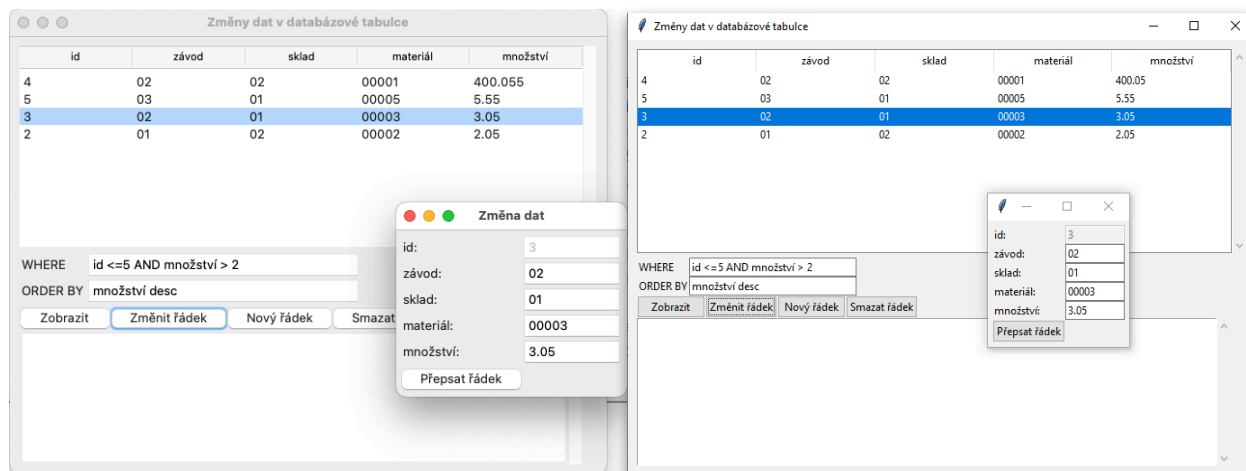
WHERE

ORDER BY

Tlačítka *Změnit řádek*, *Nový řádek*, *Smazat řádek* odpovídají příkazům UPDATE, INSERT, DELETE.

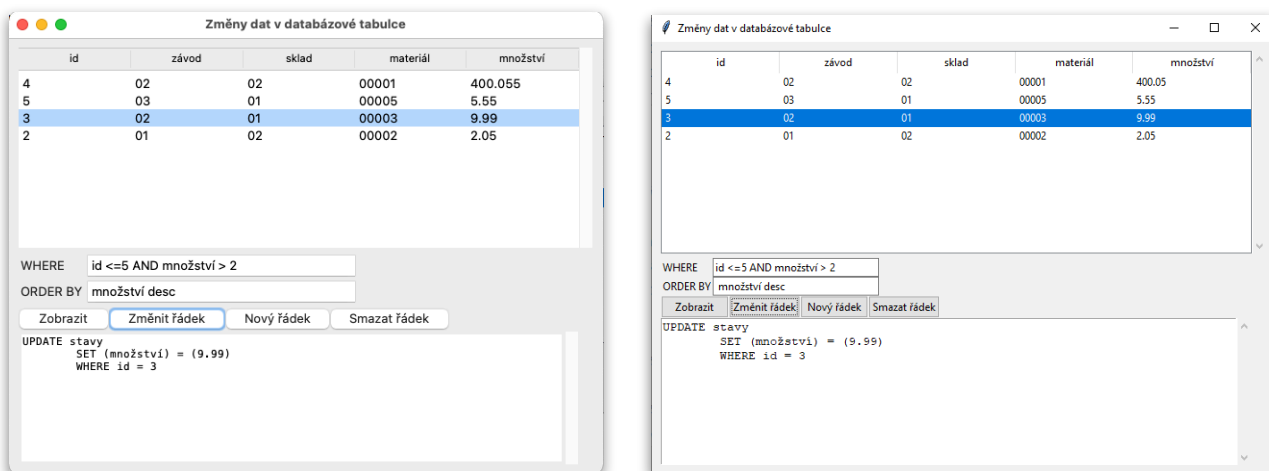
Změna řádku

Když uživatel vybere řádek s hodnotou 3 sloupce `id`, a stiskne tlačítko *Změnit řádek*, zobrazí se nové okno pro přepsání hodnot zvolených sloupců. Podobná okna se zobrazí i pro další tlačítka.



Hodnotu sloupce `id` není možné přepsat. Když uživatel přepíše některé hodnoty sloupců a stiskne tlačítko *Přepsat řádek*, provede se oprava v databázi a také v obrazové tabulce.

Například změni-li sloupec `množství`, který je číselný, z hodnoty 3.05 na hodnotu 99.99, projeví se to okamžitě i v hlavním okně. Ostatní sloupce se nezmění.



Nový řádek

Pro vložení nového řádku uživatel stiskne tlačítko *Nový řádek*.

- Když přitom *není vybrán žádný řádek*, zobrazí se podobné okno, ale se všemi hodnotami sloupců nulovými. Uživatel přepíše nuly svými hodnotami, přičemž *textové hodnoty uzavře do apostrofů* (jinak by se proměnily na čísla) a stiskne tlačítko *Vložit řádek*. Přepisuje-li i sloupec `id`, musí se trefit do čísla (zde např. 0), které v databázové tabulce neexistuje. Jinak nastane chyba integrity.
- Když je vybrán řádek, zobrazí se okno s hodnotami sloupců stejně jako v případě změny řádku, ale s povoleným přepsáním sloupce `id`.

Smazání řádku

Pro smazání řádku uživatel vybere řádek a stiskne tlačítko *Smazat řádek*. Zobrazí se dialogové okno s přehledem sloupců s dvěma tlačítky a žádá potvrdit akci nebo ji zrušit.

Hlavní část programu

Hlavní část programu je zapsána na konci a začíná importy a definicemi globálních proměnných, které potřebujeme v dalších částech programu a pokračuje zpracováním spouštěcího a hlavního okna.

```
from tkinter import *
from tkinter import ttk
import sqlite3, sys
import sqlite_utils

# globální proměnné
schema = '/Users/vzupka/PycharmProjects/tkinter/zasoby.db'
tabulka = 'stavy'
jmena_sl = [] # seznam jmen sloupců databázové tabulky
connection = sqlite3.Connection('')
cursor = sqlite3.Cursor(connection)
```

Vytvoření okna dialogu pro zadání databázové tabulky

```
dialog_zadani = Tk()
dialog_zadani.title('Zadání tabulky')
dialog_zadani.geometry('+600+300') # WxH chybí, pozici +X+Y zadáváme

# rámec pro dialog a scroll bar
ramec_zadani = ttk.Frame(dialog_zadani, padding='10px') 1
ramec_zadani.grid(column=0, row=0)

napis_schema = ttk.Label(ramec_zadani, text='Databáze (schema):')
var_schema = StringVar(value=schema) 2
text_schema = ttk.Entry(ramec_zadani, width=60, textvariable=var_schema)
```

1 Rámec třídy `Frame` bude zevnitř vyplněn pásem o šířce 10 obrazových bodů (pixels).

2 Třída `StringVar` vytváří proměnnou, do níž se uloží hodnota, kterou uživatel zapíše do vstupního pole. Tu později získáme metodou `get()`. Viz funkci `spustit()` [dále](#).

```

napis_tabulka = ttk.Label(ramec_zadani, text='Tabulka:')
var_tabulka = StringVar(value=tabulka)
text_tabulka = ttk.Entry(ramec_zadani, width=15, textvariable=var_tabulka)

tlac_spustit = ttk.Button(ramec_zadani, text='Spustit výpočet', command=spustit)
tlac_zrusit = ttk.Button(ramec_zadani, text='Zrušit', command=zrusit)

zprava = ttk.Label(ramec_zadani, text='', foreground='red4') 1

napis_schema.grid(sticky=W) 2
text_schema.grid(sticky=W)
napis_tabulka.grid(sticky=W)
text_tabulka.grid(sticky=W)
tlac_spustit.grid(sticky=W)
tlac_zrusit.grid(sticky=W)
zprava.grid(sticky=W)

dialog_zadani.protocol("WM_DELETE_WINDOW", sys.exit) # uzávěr okna dialogu 3

dialog_zadani.mainloop() # cyklus pro dialog zadání

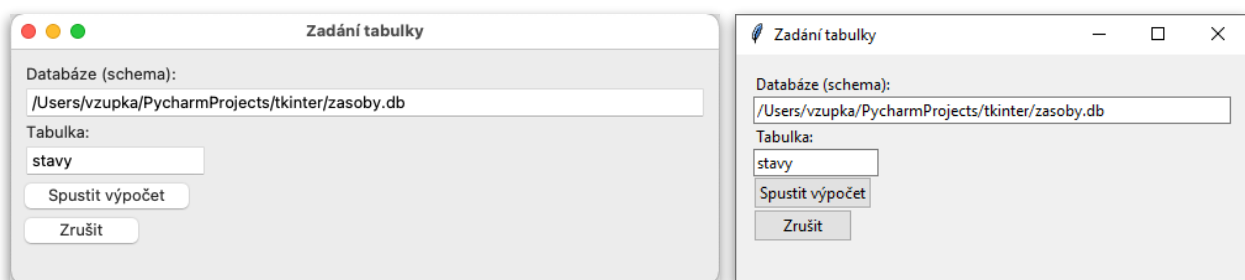
```

1 Proměnná `zprava` typu `Label` s prázdným textem a přidělenou červenou barvou je určena k zobrazení chybové zprávy v dolní části okna dialogu.

2 Prvky jsou rozvrženy metodou `grid()` v nadřazeném objektu (`ramec_zadani`) podle sloupců a řádků do mřížky, která tím vzniká. Není-li argument `column` zadán, je číslo sloupce *nula*. Není-li zadán argument `row`, je u prvního prvku nulový a u dalších prvků se automaticky zvyšuje o *jedničku*.

Parametr `sticky=W` znamená, že prvek je ve mřížce zarovnán vlevo (`W`=west, západ).

3 Klepne-li uživatel na uzávěr okna, nastane událost `WM_DELETE_WINDOW`. Metoda `protocol()` vyvolá funkci `sys.exit()` bez argumentu, která ukončí program.



Vytvoření hlavního okna

```
hlavni_okno = Tk() # vytvořím hlavní okno
hlavni_okno.title('Změny dat v databázové tabulce')
hlavni_okno.geometry('+600+300') # velikost WxH chybí, pozici +X+Y zadáváme

# rámec pro tabulku a posuvník
ramec_hlav = ttk.Frame(hlavni_okno, padding=10) # s okrajem šířky 10 bodů
ramec_hlav.grid() # umístěn do řádku 0 a sloupce 0

obraz_tabulka = ttk.Treeview(ramec_hlav, columns=jmena_sl, show='headings', 1
                             height=10)
scroll_bar_obr = ttk.Scrollbar(ramec_hlav, orient=VERTICAL) 2
obraz_tabulka.configure(yscrollcommand=scroll_bar_obr.set) 3

obraz_tabulka.grid(row=0, sticky='WE') # tabulka roztažena zleva doprava
scroll_bar_obr.grid(column=1, row=0, sticky='NS') # roztažen shora dolů

# rámec pro WHERE a ORDER BY
ramec_whord = ttk.Frame(ramec_hlav, padding=(0, 5, 0, 0)) 4
ramec_whord.grid(row=4, sticky=W) # zarovnan vlevo

napis_where = ttk.Label(ramec_whord, text='WHERE')
var_where = StringVar(value='')
text_where = ttk.Entry(ramec_whord, width=30, textvariable=var_where)

napis_orderby = ttk.Label(ramec_whord, text='ORDER BY')
var_orderby = StringVar(value='')
text_orderby = ttk.Entry(ramec_whord, width=30, textvariable=var_orderby)

napis_where.grid(column=0, row=1, sticky=W) # zarovnan vlevo
text_where.grid(column=1, row=1)
napis_orderby.grid(column=0, row=2, sticky=W) # zarovnan vlevo
text_orderby.grid(column=1, row=2)
```

1 Obrazová tabulka s hlavičkami (jmény sloupců). Má prostor pro 10 řádků. Data řádků a sloupců se vytvoří ve funkci `zobrazení_tabulky()`.

2 Posuvník orientovaný svisle.

3 Konfigurace (dodatečná modifikace) tabulky připojí pomocí metody `set()` posuvník k obrazové tabulce.

4 Parametr `padding` (výplň) určuje tloušťku pásu ubraného rámcí zevnitř. *Jedna* hodnota znamená ubrání ze všech stran, *dvě* hodnoty zleva a zprava, *čtyři* hodnoty zleva, shora, zprava a zespoda; dvě a čtyři hodnoty zapsané jako n-tice v závorce, např. `(0, 5)` a `(0, 5, 0, 0)`, jedna hodnota bez závorek.

```

# rámeček pro tlačítka
ramec_tlac = ttk.Frame(ramec_hlav)
ramec_tlac.grid(column=0, row=5, sticky=W)

tlac_zobrazit = ttk.Button(ramec_tlac, text='Zobrazit',
                           command=zobrazeni_tabulky) 1
tlac_zmenit = ttk.Button(ramec_tlac, text='Změnit řádek',
                         command=lambda: upravit('Změnit řádek')) 2
tlac_novy = ttk.Button(ramec_tlac, text='Nový řádek',
                      command=lambda: upravit('Nový řádek'))
tlac_smazat = ttk.Button(ramec_tlac, text='Smazat řádek',
                        command=lambda: upravit('Smazat řádek'))

tlac_zobrazit.grid(column=0, row=3)
tlac_zmenit.grid(column=1, row=3)
tlac_novy.grid(column=2, row=3)
tlac_smazat.grid(column=3, row=3)

# rámeček pro log
ramec_log = ttk.Frame(ramec_hlav)
ramec_log.grid(column=0, row=6)

log = Text(ramec_log, height=10) # okno logu pro 10 řádků
log.tag_configure(tagName='error', foreground='red4') 3
log.grid(column=0, row=1)

scroll_bar_log = ttk.Scrollbar(ramec_log, orient=VERTICAL)
log.configure(yscrollcommand=scroll_bar_log.set) # přidání posuvníku k logu
scroll_bar_log.grid(column=1, row=1, sticky='NS') # roztažen shora dolů

zobrazeni_tabulky() # vyvolám první zobrazení tabulky

hlavni_okno.mainloop()

```

- 1 Tlačítko vyvolá funkci `zobrazeni_tabulky()`.
- 2 Ostatní tlačítka volají stejnou funkci `upravit()`, ale předávají jí text, podle něhož pozná, ze kterého tlačítka byla vyvolána. K předání argumentu je použito nepřímého volání pomocí anonymní funkce `lambda`. Zadání pouhého jména funkce (jako třeba v bodu 1) neumožňuje zadat argument volání, který je zapotřebí k rozlišení výpočtu. Jinak bychom museli pro každé tlačítko vytvořit samostatnou a skoro stejnou funkci.
- 3 Metoda `tag_configure()` dává textovému objektu `log` červenou barvu a příznak (tag) `'error'`, na který se v různých částech programu odvolává příkaz `log.insert()` pro zápis chyby.

Vytvoření dialogu pro změnu záznamů

Následují příkazy k vytvoření okna dialogu, které jsou *součástí funkce* `upravit()` volané z hlavního okna s argumentem `tlac_upravit`. Kromě vytvoření okna jsou součástí této funkce ještě vnořené funkce pro zpracování změn – `prepsat()`, `vlozit()`, `opravdu_smazat()` a `zrusit_mazani()`. Ty budou probírány později.

```
db = sqlite_utils.Database(connection) # přístup k údajům o databázi
# získám slovník typů sloupců tabulky ve tvaru <class 'str'>, <class 'int'> atd.
typy_sloupcu = db[tabelka].columns_dict

dialog = Tk()
dialog.title('Změna dat')
dialog.geometry('+1000+500') # velikost WxH chybí, pozici +X+Y zadáváme

# vytvořím rámec s prvky dialogu pro změnu
ramec = ttk.Frame(dialog, padding=5) # výplň 5 bodů kolem rámce
ramec.grid() # řádek 0, sloupec 0

col_id_db = jmena_sl.index("id") # číslo sloupce "id" v databázi 1

iids = obraz_tabelka.selection() # n-tice řádků, např. ('I001', 'I003')
row = {} # slovník pro data v řádku tabulky
if iids: # jsou-li vybrány nějaké řádky,
    iid = iids[0] # vyberu jen první z vybraných 2
    row = obraz_tabelka.set(iid) # a naplním slovník daty 3
    # row = {'id': '1', 'závod': '01', ...}
else: # není-li vybrán řádek u změny nebo smazání, končíme 4
    if tlac_upravit == "Změnit řádek" or tlac_upravit == "Smazat řádek":
        print("Není vybrán řádek.")
        dialog.destroy() # zničím okno
        return # návrat do hlavního okna
```

1 Číslo sloupce "id" v seznamu jmen sloupců určím pomocí metody `index()`. Bude potřeba pro zjištění hodnoty tohoto sloupce.

2 proměnná `iid` teď obsahuje *identifikátor* sloupce tabulky, např. I008.

3 Slovník `row` se naplní klíči a hodnotami pomocí funkce `set(iid)` z řádku tabulky, např. `{'id': '1', 'závod': '01', ...}`. Bude ho zapotřebí k plnění textových polí.

4 Jestliže nebyl vybrán žádný řádek, pak u *Změny* a *Smazání* o tom ohlásím zprávu (jen do systémového výstupu), zničím okno a ukončím funkci `upravit()` návratem do hlavního okna.

```

# sestavím formulář s názvy sloupců a textových polí s hodnotami.
# textové pole u sloupce "id" bude možné přepsat jen při vkládání nového řádku.
text_pole = [] # seznam textových polí
stare_hodnoty_poli = [] # vyprázdním seznam starých hodnot

for col_nbr in range(len(jmena_sl)): # cyklus podle sloupců
    # nápisy jmen sloupců
    napis = ttk.Label(ramec, text=jmena_sl[col_nbr] + ": ")
    napis.grid(column=0, row=col_nbr, sticky=W)
    # textová pole
    text_field = ttk.Entry(ramec, width=10)
    if tlac_upravit == 'Nový řádek' and not iids: # nový řádek bez výběru
        text_field.insert(0, '0') # dosadím znakovou nulu
    else:
        text_field.insert(0, row[jmena_sl[col_nbr]]) # hodnota ze slovníku 1
        if tlac_upravit == 'Smazat řádek':
            text_field.config(state='disabled')
        if col_nbr == col_id_db:
            val_id = row[jmena_sl[col_nbr]] # hodnota sloupce "id"
            if tlac_upravit == "Změnit řádek":
                text_field.config(state='disabled') # zákaz zápisu do klíče 2
    text_field.grid(column=1, row=col_nbr)

# vkládaná tlačítka se liší podle argumentu a volají příslušné funkce
if tlac_upravit == 'Změnit řádek':
    tlac_prepsat = ttk.Button(ramec, text="Přepsat řádek",
                              command=prepsat)
    tlac_prepsat.grid(column=0, row=len(jmena_sl)+1)
if tlac_upravit == 'Nový řádek':
    tlac_vlozit = ttk.Button(ramec, text="Vložit řádek",
                              command=vlozit)
    tlac_vlozit.grid(column=0, row=len(jmena_sl)+1)
if tlac_upravit == 'Smazat řádek':
    tlac_smazat = ttk.Button(ramec, text="Opravdu smazat",
                              command=opravdu_smazat)
    tlac_smazat.grid(column=0, row=len(jmena_sl)+1)
    tlac_zrusit = ttk.Button(ramec, text="Zrušit mazání",
                              command=zrusit_mazani)
    tlac_zrusit.grid(column=0, row=len(jmena_sl)+2)

field_value = text_field.get() # vezmu hodnotu z textového pole 3
stare_hodnoty_poli.append(field_value) # přidám hodnotu do seznamu hodnot
text_pole.append(text_field) # přidám textové pole do seznamu

```

1 Do textového pole (od pozice 0) se vloží metodou `insert()` hodnota sloupce ze slovníku, který byl již naplněn daty. Slovník obsahuje údaje z vybraného řádku tabulky, např. `{'id': '1', 'závod': '01', ...}`.

2 Sloupec "id" nesmí být přepisován, proto se pomocí metody `config()` znemožní zápis.

3 Hodnota textového pole se získá metodou `get()` a přidá se do seznamu dosavadních (starých) hodnot z vybraného řádku. Rovněž objekt textového pole se přidá do seznamu textových polí. Oba seznamy budou potřebné pro porovnání, zda uživatel některé hodnoty změnil nebo ne.

Výsledkem jsou okna tohoto tvaru:

macOS

The image shows three macOS-style dialog boxes, each titled "Změna dat". Each dialog contains five text input fields with labels: "id:", "závod:", "sklad:", "materiál:", and "množství:". The values entered in the fields are: "id:" is 1, "závod:" is 01, "sklad:" is 01, "materiál:" is 00009, and "množství:" is 1.05. The first dialog has a button labeled "Přepsat řádek". The second dialog has a button labeled "Vložit řádek". The third dialog has two buttons: "Opravdu smazat" and "Zrušit mazání".

Windows

The image shows three Windows-style dialog boxes, each titled "Změna dat". Each dialog contains five text input fields with labels: "id:", "závod:", "sklad:", "materiál:", and "množství:". The values entered in the fields are: "id:" is 1, "závod:" is 01, "sklad:" is 01, "materiál:" is 00009, and "množství:" is 1.05. The first dialog has a button labeled "Přepsat řádek". The second dialog has a button labeled "Vložit řádek". The third dialog has two buttons: "Opravdu smazat" and "Zrušit mazání".

Funkce tlačítek

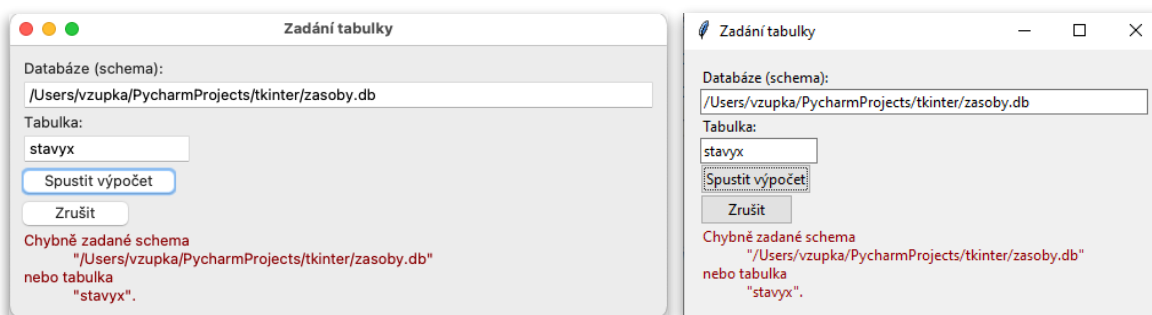
Tlačítka dialogu zadání databáze

Tlačítko "Spustit výpočet" zkontroluje, zda údaje o schematu a tabulky jsou správné (zda objekty existují) a v případě chyby ohlásí zprávu `zprava` jako statický text na spodní okraj dialogového okna.

```
def spustit():
    global jmena_sl, connection, cursor
    global schema, tabulka, jmena_sl
    schema = var_schema.get()
    tabulka = var_tabulka.get()
    try:
        connection = sqlite3.connect(schema) # připojím databázi (schema)
        cursor = connection.cursor() # definuji objekt typu Cursor
        text_prikazu = f'PRAGMA table_info({tabulka})' # zjistím sloupce tabulky
        sloupce_info = cursor.execute(text_prikazu).fetchall()
        jmena_sl = [col[1] for col in sloupce_info] # seznam jmen sloupců
        if not jmena_sl:
            error = f'''Chybně zadané schema
"{schema}"
nebo tabulka
"{tabulka}".'''
            zprava.configure(text=error) # zápis chyby 1
        else:
            dialog_zadani.destroy() # když není chyba, smažu dialog a pokračuji

    except sqlite3.Error as err:
        error = f'''Chybně zadané schema "{schema}"
nebo tabulka "{tabulka}".'''
        zprava.configure(text=error) # zápis chyby 1
```

1 Metoda `configure()` (nebo `config`) modifikuje statický text `zprava` (typ `ttk.Label`) vložením chybové zprávy `error`, což způsobí její zobrazení v okně. Jindy je text zprávy prázdný.



Tlačítko "**Zrušit**" zničí dialog a opustí program.

```
def zrusit():  
    dialog_zadani.destroy() # smažu okno dialogu zadání  
    sys.exit()
```

Tlačítka hlavního okna

Tlačítko "Zobrazit" vyvolá funkci `zobrazeni_tabulky()`, ostatní tlačítka volají stejnou funkci `upravit()`, která rozlišuje, které tlačítko ji vyvolalo.

Funkce `zobrazeni_tabulky()` přečte texty z polí pro SQL výrazy WHERE a ORDER BY, sestaví a pak provede příkaz SELECT. Pak údaje z výsledné tabulky zapíše do obrazové tabulky.

```
def zobrazeni_tabulky():
    where = text_where.get()
    print(where)
    orderby = text_orderby.get()

    text_prikazu = f'SELECT {", ".join(jmena_sl)} FROM {tabulka}'
    if where:
        text_prikazu += f' WHERE {where} '
    if orderby:
        text_prikazu += f' ORDER BY {orderby} '
    text_prikazu += ' LIMIT 1000'
    try:
        cursor.execute(text_prikazu) # provedu dotaz
        # smažu obsah obrazové tabulky
        obraz_tabulka.delete(* obraz_tabulka.get_children()) 1
        zaznamy = cursor.fetchall() # získám seznam záznamů
        # zapíšu hodnoty ze záznamů do obrazové tabulky
        [obraz_tabulka.insert("", 'end', values=zaznam)
         for zaznam in zaznamy] # hodnoty sloupců 2
        [obraz_tabulka.heading(col_nbr, text=jmena_sl[col_nbr])
         for col_nbr in range(len(jmena_sl))] # hlavičky 3
        [obraz_tabulka.column(col_nbr, width=80, anchor=W)
         for col_nbr in range(len(jmena_sl))] # šířky sloupců 4
    except sqlite3.Error as err:
        print(f"Chyba příkazu: {err}")
        log.insert('end', f"Chyba příkazu: {err}\n", 'error') 5
```

1 Hvězdička znamená, že argument je vícenásobný (variadic). Metoda `delete()` smaže všechny podřízené objekty (children) a vyprázdní tedy tabulku.

2 Příkaz vloží hodnoty databázových záznamů do řádků tabulky. Záznamy tvoří dvojrozměrný seznam seznamů sloupců. Výraz v hranatých závorkách se nazývá "list comprehension" a představuje cyklus, který probíhá přes všechny záznamy (seznamy sloupců).

3 Příkaz zařadí hlavičky sloupců do tabulky.

4 Příkaz určuje šířky sloupců v tabulce (jednotně 80 bodů).

5 Metoda `insert()` zapíše další řádek do logu (objekt typu `ttk.Text`). Konec řádku `'\n'` musí být výslovně zadán. Argument `'error'` spojuje příkaz s hodnotou argumentu `tagName` v příkazu `log.tag_configure()` zapsaném v hlavním programu.

Tlačítka **Změnit řádek**, **Nový řádek**, **Smazat řádek** volají stejnou funkci `upravit()`, která rozlišuje, které tlačítko ji vyvolalo.

Funkce **`upravit(tlacitko_upravit)`** kromě sestavení dialogového okna (které již bylo popsáno) obsahuje ještě *vnitřní funkce*, `prepsat()`, `vlozit()`, `opravdu_smazat()` a `zrusit_mazani()` vyvolávané podle názvu tlačítka, který je vstupním argumentem.

Funkce **`prepsat()`** - činnost tlačítka **"Změnit řádek"**

```
def prepsat():
    hodnoty = [] # seznam hodnot pro UPDATE SET ( sloupce ) = ( hodnoty )
    sloupce = jmena_sl.copy() # kopie seznamu sloupců
    sloupce.remove("id") # odstraním sloupec "id" z kopie
    # sestavím seznam hodnot z textových polí
    selected = obraz_tabulka.focus() 1
    nove_hodnoty = [] # seznam nových hodnot pro obrazovou tabulku
    for col_nbr in range(len(jmena_sl)):
        if jmena_sl[col_nbr] != "id": # bez sloupce "id"
            nova_hodnota = text_pole[col_nbr].get()
            stara_hodnota = stare_hodnoty_poli[col_nbr]
            if nova_hodnota == stara_hodnota: # vynechám nezměněné sloupce
                sloupce.remove(jmena_sl[col_nbr]) # odstraním jméno ze seznamu 2
            else: # do hodnot přidám text
                # textové hodnoty (typu <class 'str'>) musí být v apostrofech
                if repr(typy_sloupcu[jmena_sl[col_nbr]]) == "<class 'str'>": 3
                    hodnoty.append(f"'{text_pole[col_nbr].get()}'") # s apostrofy
                else:
                    hodnoty.append(f"{text_pole[col_nbr].get()}") # bez apostrofů 4
            nove_hodnoty.append(text_pole[col_nbr].get()) # text z pole do nových hodnot
    if sloupce: # a není-li seznam sloupců prázdný
        text_prikazu = f"UPDATE {tabulka}
SET ({', '.join(sloupce)}) = ({', '.join(hodnoty)})
WHERE id = {val_id}" # záznam je určen jedinečným údajem hodnoty sloupce id
        print(text_prikazu)
        log.insert(END, text_prikazu + '\n') # zapíšu příkaz do logu
        try:
            cursor.execute(text_prikazu)
            connection.commit()
            log.insert(END, text_prikazu + '\n') # zapíšu příkaz do logu
            # v obrazové tabulce se aktualizuje jen vybraný řádek!
            obraz_tabulka.item(selected, values=nove_hodnoty). 5
        except sqlite3.Error as err:
            print(f"Chyba: {err}")
            log.insert(END, f"Chyba: {err}\n", 'error')
    else:
        print("Záznam se nemění.")
    dialog.destroy() # návrat do hlavního okna
```

1 U změny řádku se porovnávají nové hodnoty vložené uživatelem do textových polí s jejich dosavadními hodnotami uloženými dříve do seznamu `stare_hodnoty_poli`.

2 Nezměnila-li se hodnota, odstraním jméno příslušného sloupce ze seznamu určeného pro příkaz UPDATE.

3 Změnila-li se hodnota, a přidá se do seznamu hodnot pro příkaz UPDATE. Je-li její typ `str` (text), uzavře se do apostrofů.

4 Hodnota textového pole se přidá také do seznamu nových hodnot pro obrazovou tabulku.

5 Metoda `item()` vloží nové hodnoty jen do vybraného řádku obrazové tabulky.

Funkce **vlozit()** – činnost tlačítka "Vložit řádek"

```
def vlozit():
    hodnoty = [] # seznam hodnot pro INSERT ... VALUES ( hodnoty )
    for col_nbr in range(len(jmena_sl)): # hodnoty z textových polí
        # textové hodnoty (typu <class 'str'>) musí být v apostrofech
        if repr(typy_sloupcu[jmena_sl[col_nbr]]) == "<class 'str'>":
            hodnoty.append(f"'{text_pole[col_nbr].get()}'") # text s apostrofy
        else:
            hodnoty.append(f"{text_pole[col_nbr].get()}") # text bez apostrofů
    text_prikazu = f"INSERT INTO {tabulka} ({', '.join(jmena_sl)}) VALUES ({', '.join(hodnoty)})"
    print(text_prikazu)
    log.insert(END, text_prikazu + '\n') # zápis do logu
    try:
        cursor.execute(text_prikazu)
        connection.commit()
        zobrazeni_tabulky() # zobrazím SQL tabulku v obrazové tabulce znovu 1
    except sqlite3.Error as err:
        print(f"Chyba: {err}")
        log.insert(END, f"Chyba: {err}\n", 'error')
```

Funkce **opravdu_smazat()** – činnost tlačítka "Opravdu smazat"

```
def opravdu_smazat():
    # sestavím a provedu SQL DELETE příkaz pro smazání záznamu
    text_prikazu = f"DELETE FROM {tabulka} WHERE id = {val_id}" 2
    print(text_prikazu)
    log.insert(END, text_prikazu + '\n') # zápis do logu

    cursor.execute(text_prikazu) # příkaz DELETE smaže řádek z databáze
    connection.commit() # musím potvrdit transakci, jinak se příkaz neprovede
    selected = obraz_tabulka.focus() # # vrátí iid právě vybraného řádku 3
    obraz_tabulka.delete(selected) # smaže vybraný řádek z tabulky
    dialog.destroy() # návrat do hlavního okna
    # okno zůstává otevřené
```

Funkce **zrusit_mazani()** – činnost tlačítka "Zrušit mazání"

```
def zrusit_mazani():
    dialog.destroy() # návrat do hlavního okna
```

1 Po přidání záznamu do databáze musím tabulku znovu zobrazit.

2 V proměnné `val_id` je jedinečná hodnota sloupce "id" z vybraného řádku, podle níž příkaz `DELETE` smaže záznam.

3 Metoda `focus()` bez argumentu vrátí identifikátor (iid) právě vybraného řádku tabulky. Identifikátor má tvar `Innn`, kde `nnn` je číslo řádku. V následujícím příkazu metoda `delete()` smaže příslušný řádek tabulky (bez následného zobrazení jako při vkládání).

Celý program

```
# funkce tlačítek dialog_zadani k zadání schematu a tabulky
# -----
def spustit():
    global jmena_sl, connection, cursor
    global schema, tabulka, jmena_sl
    schema = var_schema.get()
    tabulka = var_tabulka.get()
    try:
        connection = sqlite3.connect(schema) # připojím databázi (schema)
        cursor = connection.cursor() # definuji objekt typu Cursor
        text_prikazu = f'PRAGMA table_info({tabulka})' # zjistím sloupce tabulky
        sloupce_info = cursor.execute(text_prikazu).fetchall()
        jmena_sl = [col[1] for col in sloupce_info] # seznam jmen sloupců
        if not jmena_sl:
            error = f'''Chybně zadané schema
            "{schema}"
nebo tabulka
            "{tabulka}".'''
            zprava.configure(text=error) # změna statického textu v okně
        else:
            dialog_zadani.destroy() # když není chyba, smažu dialog a pokračuji
    except sqlite3.Error as err:
        error = f'''Chybně zadané schema "{schema}"
nebo tabulka "{tabulka}".'''
        zprava.configure(text=error) # zápis do logu

def zrusit():
    dialog_zadani.destroy() # smažu okno dialogu zadání
    sys.exit()
# -----

# funkce zobrazení databázové tabulky v obrazové tabulce
# -----
def zobrazeni_tabulky():
    global obrazova_tabulka
    where = text_where.get()
    print(where)
    orderby = text_orderby.get()

    text_prikazu = f'SELECT {"", ".join(jmena_sl)} FROM {tabulka}'
    if where:
        text_prikazu += f' WHERE {where} '
    if orderby:
        text_prikazu += f' ORDER BY {orderby} '
    text_prikazu += ' LIMIT 1000'
    try:
        cursor.execute(text_prikazu) # provedu dotaz
        # smažu obsah obrazové tabulky
        obraz_tabulka.delete(* obraz_tabulka.get_children()) # násobné (variadic) argumenty
        zaznamy = cursor.fetchall() # získám seznam záznamů
        # zapíšu hodnoty ze záznamů do obrazové tabulky
        [obraz_tabulka.insert('', 'end', values=zaznam) for zaznam in zaznamy]
        [obraz_tabulka.heading(col_nbr, text=jmena_sl[col_nbr])
         for col_nbr in range(len(jmena_sl))] # hlavičky
        [obraz_tabulka.column(col_nbr, width=80, anchor=W)
         for col_nbr in range(len(jmena_sl))]
    except sqlite3.Error as err:
        print(f"Chyba příkazu: {err}")
        log.insert(END, f"Chyba příkazu: {err}" + '\n', 'error') # zápis do logu
# -----

# funkce dialogu pro změnu záznamů
# -----
def upravit(tlac_upravit):
    global jmena_sl
```

```

# činnost tlačítka "Přepsat řádek"
# -----
def prepsat(): # je vybrán řádek
    hodnoty = [] # seznam hodnot pro UPDATE SET ( sloupce ) = ( hodnoty )
    sloupce = jmena_sl.copy() # kopie seznamu sloupců pro UPDATE SET ( sloupce ) = ( hodnoty )
    sloupce.remove("id") # odstraním sloupec "id" z kopie
    # sestavím seznam hodnot z textových polí
    selected = obraz_tabulka.focus()
    nove_hodnoty = [] # seznam nových hodnot pro obrazovou tabulku
    for col_nbr in range(len(jmena_sl)):
        if jmena_sl[col_nbr] != "id": # bez sloupce "id"
            nova_hodnota = text_pole[col_nbr].get()
            stara_hodnota = stare_hodnoty_poli[col_nbr]
            if nova_hodnota == stara_hodnota: # vynechám nezměněné sloupce
                sloupce.remove(jmena_sl[col_nbr]) # odstraním jméno ze seznamu pro UPDATE
            else: # do hodnot přidám text
                # textové hodnoty (typu <class 'str'>) musí být v apostrofech
                if repr(typy_sloupcu[jmena_sl[col_nbr]]) == "<class 'str'>":
                    hodnoty.append(f"'{text_pole[col_nbr].get()}'") # text s apostrofy
                else:
                    hodnoty.append(f"{text_pole[col_nbr].get()}") # text bez apostrofů
            nove_hodnoty.append(text_pole[col_nbr].get()) # přidám text z pole do nových hodnot

    if sloupce: # a není-li seznam sloupců prázdný
        text_prikazu = f"UPDATE {tabulka}
SET ({', '.join(sloupce)}) = ({', '.join(hodnoty)})
WHERE id = {val_id}" # záznam je určen jedinečným údajem hodnoty id
        print(text_prikazu)
        try:
            cursor.execute(text_prikazu)
            connection.commit()
            # v obrazové tabulce se aktualizuje jen vybraný řádek!
            obraz_tabulka.item(selected, values=nove_hodnoty) # aktualizuji vybraný řádek
            log.insert(END, text_prikazu + '\n') # zapíšu příkaz do logu
        except sqlite3.Error as err:
            print(f"Chyba: {err}")
            log.insert(END, f"Chyba: {err}\n", 'error')
    else:
        print("Záznam se nemění.")
        log.insert(END, "Záznam se nemění.\n") # zapíšu příkaz do logu
    dialog.destroy() # zničím dialogové okno a vrátím se do hlavního okna

# -----

# činnost tlačítka "Nový řádek"
# -----
def vlozit():
    hodnoty = [] # seznam hodnot pro INSERT ... VALUES ( hodnoty )
    for col_nbr in range(len(jmena_sl)): # hodnoty z textových polí
        # textové hodnoty (typu <class 'str'>) musí být v apostrofech
        if repr(typy_sloupcu[jmena_sl[col_nbr]]) == "<class 'str'>":
            hodnoty.append(f"'{text_pole[col_nbr].get()}'") # text s apostrofy
        else:
            hodnoty.append(f"{text_pole[col_nbr].get()}") # text bez apostrofů
    text_prikazu = f"INSERT INTO {tabulka}
({', '.join(jmena_sl)}) VALUES ({', '.join(hodnoty)})"
    print(text_prikazu)
    log.insert(END, text_prikazu + '\n') # zápis do logu
    try:
        cursor.execute(text_prikazu)
        connection.commit()
        zobrazeni_tabulky() # zobrazím SQL tabulku v obrazové tabulce znovu
    except sqlite3.Error as err:
        print(f"Chyba: {err}")
        log.insert(END, f"Chyba: {err}\n", 'error')
    # okno zůstává otevřené

# -----

# činnost tlačítka "Opravdu smazat"
# -----
def opravdu_smazat():
    # sestavím a provedu SQL DELETE příkaz pro smazání záznamu

```

```

text_prikazu = f"DELETE FROM {tabulka} WHERE id = {val_id}"
print(text_prikazu)

cursor.execute(text_prikazu) # příkaz DELETE smaže řádek z databáze
connection.commit() # musím potvrdit transakci, jinak se příkaz neprovede
selected = obraz_tabulka.focus() # vrátí iid právě vybraného řádku
obraz_tabulka.delete(selected) # smaže vybraný řádek z obrazové tabulky
log.insert(END, text_prikazu + '\n') # zápis do logu
dialog.destroy() # zničím dialogové okno a vrátím se do hlavního okna

# -----

# činnost tlačítka "Zrušit mazání"
# -----
def zrusit_mazani():
    dialog.destroy() # zničím dialogové okno a vrátím se do hlavního okna
# -----

#####
# 4 #
#####
# dialog pro změnu záznamů
# -----
db = sqlite_utils.Database(connection) # přístup k údajům o databázi
# získám slovník typů sloupců tabulky ve tvaru <class 'str'>, <class 'int'> atd.
typy_sloupcu = db[tabulka].columns_dict

dialog = Tk()
dialog.title('Změna dat')
dialog.geometry('+1000+500') # velikost WxH je standardní(chybí), pozici +X+Y zadáváme

# vytvořím rámec s prvky dialogu pro změnu
ramec = ttk.Frame(dialog, padding=5) # výplň 5 bodů kolem rámce
ramec.grid() # řádek 0, sloupec 0

col_id_db = jmena_sl.index("id") # číslo sloupce "id" v databázi

iids = obraz_tabulka.selection() # n-tice identifikátorů vybraných řádků, např. ('I001',
'I003')
row = {} # slovník pro data v řádku tabulky
if iids: # jsou-li vybrány nějaké řádky,
    iid = iids[0] # vyberu jen první z vybraných
    row = obraz_tabulka.set(iid) # a naplním slovník daty
else: # není-li vybrán řádek u změny nebo smazání, končíme
    if tlac_upravit == "Změnit řádek" or tlac_upravit == "Smazat řádek":
        print("Není vybrán řádek.")
        dialog.destroy() # zničím okno
        return # návrat do hlavního okna

# sestavím formulář s názvy sloupců a textových polí s hodnotami.
# textové pole u sloupce "id" bude možné přepsat jen při vkládání nového řádku.
text_pole = []
stare_hodnoty_poli = [] # vyprázdním seznam starých hodnot

for col_nbr in range(len(jmena_sl)):
    # nápisy jmen sloupců
    napis = ttk.Label(ramec, text=jmena_sl[col_nbr] + ": ")
    napis.grid(column=0, row=col_nbr, sticky=W)
    # textová pole
    text_field = ttk.Entry(ramec, width=10)
    if tlac_upravit == 'Nový řádek' and not iids: # nový řádek bez výběru
        text_field.insert(0, '0') # dosadím znakovou nulu
    else:
        text_field.insert(0, row[jmena_sl[col_nbr]]) # hodnota ze slovníku řádku
        if tlac_upravit == 'Smazat řádek':
            text_field.config(state='disabled')
        if col_nbr == col_id_db:
            val_id = row[jmena_sl[col_nbr]] # hodnota databázového klíče záznamu
            if tlac_upravit == "Změnit řádek":
                text_field.config(state='disabled') # zákaz zápisu do klíčového pole
        text_field.grid(column=1, row=col_nbr)

```

```

# vkládaná tlačítka se liší podle argumentu funkce
if tlac_upravit == 'Změnit řádek':
    tlac_přepsat = ttk.Button(ramec, text="Přepsat řádek",
                             command=přepsat)
    tlac_přepsat.grid(row=len(jmena_sl)+1)
if tlac_upravit == 'Nový řádek':
    tlac_vložit = ttk.Button(ramec, text="Vložit řádek",
                             command=vložit)
    tlac_vložit.grid(row=len(jmena_sl)+1)
if tlac_upravit == 'Smazat řádek':
    tlac_smazat = ttk.Button(ramec, text="Opravdu smazat",
                             command=opravdu_smazat)
    tlac_smazat.grid(row=len(jmena_sl)+1)
    tlac_zrusit = ttk.Button(ramec, text="Zrušit mazání",
                             command=zrusit_mazani)
    tlac_zrusit.grid(row=len(jmena_sl)+2)

field_value = text_field.get() # vezmu hodnotu z textového pole
stare_hodnoty_poli.append(field_value) # přidám hodnotu do seznamu hodnot
text_pole.append(text_field) # přidám textové pole do seznamu textových polí
# -----

```

```

# -----
if __name__ == "__main__":
    #####
    # 1 #
    #####
    from tkinter import *
    from tkinter import ttk
    import sqlite3, sys
    import sqlite_utils

    # globální proměnné
    schema = '/Users/vzupka/PycharmProjects/tkinter/zasoby.db'
    tabulka = 'stavy'
    jmena_sl = [] # seznam jmen sloupců databázové tabulky
    connection = sqlite3.Connection('')
    cursor = sqlite3.Cursor(connection)

    #####
    # 2 #
    #####
    # vytvoření okna dialogu pro zadání databázové tabulky
    # -----
    dialog_zadani = Tk()
    dialog_zadani.title('Zadání tabulky')
    dialog_zadani.geometry('+600+300') # velikost WxH je standardní(chybí), pozici +X+Y zadáváme

    ramec_zadani = ttk.Frame(dialog_zadani, padding='10px') # rámeček pro hlavní okno a scroll bar
    ramec_zadani.grid(column=0, row=0)

    napis_schema = ttk.Label(ramec_zadani, text='Databáze (schema):')

    var_schema = StringVar(value=schema)
    text_schema = ttk.Entry(ramec_zadani, width=60,
                             textvariable=var_schema)

    napis_tabulka = ttk.Label(ramec_zadani, text='Tabulka:')
    var_tabulka = StringVar(value=tabulka)
    text_tabulka = ttk.Entry(ramec_zadani, width=15,
                             textvariable=var_tabulka)

    tlac_spustit = ttk.Button(ramec_zadani, text='Spustit výpočet',
                             command=spustit)
    tlac_zrusit = ttk.Button(ramec_zadani, text='Zrušit',
                             command=zrusit)
    zprava = ttk.Label(ramec_zadani, text='', foreground='red4')

    napis_schema.grid(sticky=W)
    text_schema.grid(sticky=W)
    napis_tabulka.grid(sticky=W)

```

```

text_tabulka.grid(sticky=W)
tlac_spustit.grid(sticky=W)
tlac_zrusit.grid(sticky=W)
zprava.grid(sticky=W)

dialog_zadani.protocol("WM_DELETE_WINDOW", sys.exit) # uzávěr okna dialogu

dialog_zadani.mainloop() # cyklus pro dialog zadání
# -----

#####
# 3 #
#####
# vytvoření hlavního okna
# -----
hlavni_okno = Tk() # vytvořím hlavní okno
hlavni_okno.title('Změny dat v databázové tabulce')
hlavni_okno.geometry('+600+300') # velikost WxH je standardní(chybí), pozici +X+Y zadáváme

# rámec pro tabulku a posuvník
ramec_hlav = ttk.Frame(hlavni_okno, padding=10) # s okrajem šířky 10 bodů
ramec_hlav.grid() # umístěn do řádku 0 a sloupce 0

obraz_tabulka = ttk.Treeview(ramec_hlav, columns=jmena_sl, show='headings',
                             height=10)
scroll_bar_obr = ttk.Scrollbar(ramec_hlav, orient=VERTICAL,
                              command=obraz_tabulka.yview) # přidání posuvníku k tabulce
obraz_tabulka.configure(yscrollcommand=scroll_bar_obr.set)

obraz_tabulka.grid(row=0, sticky='WE') # tabulka roztažena zleva doprava
scroll_bar_obr.grid(column=1, row=0, sticky='NS') # posuvník roztažen shora dolů

ramec_whord = ttk.Frame(ramec_hlav, padding=(0, 5, 0, 0)) # rámec pro zadání WHERE a ORDER BY
ramec_whord.grid(row=4, sticky=W) # zarovnan vlevo

napis_where = ttk.Label(ramec_whord, text='WHERE')
var_where = StringVar(value='')
text_where = ttk.Entry(ramec_whord, width=30, textvariable=var_where)

napis_orderby = ttk.Label(ramec_whord, text='ORDER BY')
var_orderby = StringVar(value='')
text_orderby = ttk.Entry(ramec_whord, width=30, textvariable=var_orderby)

napis_where.grid(column=0, row=1, sticky=W) # nápis WHERE zarovnan vlevo
text_where.grid(column=1, row=1)
napis_orderby.grid(column=0, row=2, sticky=W)
text_orderby.grid(column=1, row=2)

ramec_tlac = ttk.Frame(ramec_hlav) # rámec pro tlačítka
ramec_tlac.grid(column=0, row=5, sticky=W)

tlac_zobrazit = ttk.Button(ramec_tlac, text='Zobrazit',
                          command=zobrazeni_tabulky)
tlac_zmenit = ttk.Button(ramec_tlac, text='Změnit řádek',
                        command=lambda: upravit('Změnit řádek'))
tlac_novy = ttk.Button(ramec_tlac, text='Nový řádek',
                      command=lambda: upravit('Nový řádek'))
tlac_smazat = ttk.Button(ramec_tlac, text='Smazat řádek',
                        command=lambda: upravit('Smazat řádek'))

tlac_zobrazit.grid(column=0, row=3)
tlac_zmenit.grid(column=1, row=3)
tlac_novy.grid(column=2, row=3)
tlac_smazat.grid(column=3, row=3)

ramec_log = ttk.Frame(ramec_hlav) # rámec pro log
ramec_log.grid(column=0, row=6)

log = Text(ramec_log, height=10) # okno logu pro 10 řádků
log.tag_configure('error', foreground='red4')
log.grid(column=0, row=1)

```

```

scroll_bar_log = ttk.Scrollbar(ramec_log, orient=VERTICAL, command=log.yview)
log.configure(yscrollcommand=scroll_bar_log.set) # přidání posuvníku k logu
scroll_bar_log.grid(column=1, row=1, sticky='NS') # posuvník roztažen shora dolů

# zobrazím databázovou tabulku v obrazové tabulce
# -----
zobrazeni_tabulky() # vyvolám první zobrazení tabulky

hlavni_okno.mainloop()

```

GUI PySimpleGUI

Instaluje se příkazem

```
pip3 install pysimplegui
```

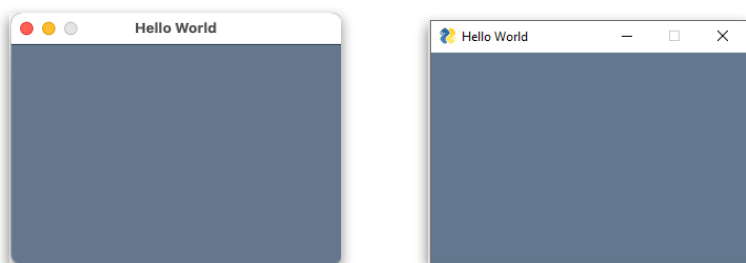
Minimální program

Minimální program Hello World vypadá takto:

```
import PySimpleGUI as sg

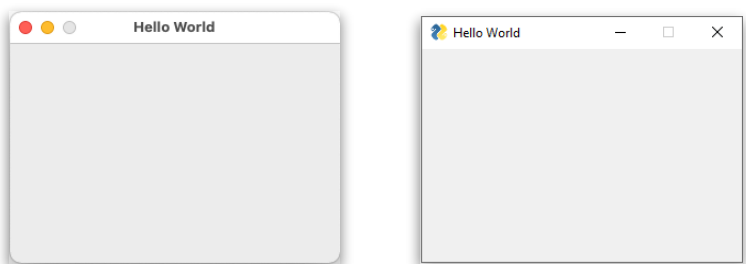
window = sg.Window('Hello World', layout=[[]], size=(300,200))
window.read()
```

Program zobrazí prázdné okno (macOS, Windows):



1. Třída `Window` představuje okno a obsahuje řadu parametrů (podobně jako třída `Frame` v balíku *wxPython*, ale mnohem více).
2. Parametr `layout` stanoví rozvrh prvků v okně ve formě seznamu seznamů, což představuje řádky a v nich prvky. V tomto příkladu je rozvrh prázdný `[]` (bez řádků a prvků v nich), ale musí být zadán.
3. Také velikost `size` musí být zadána jako dvojice šířky a výšky v obrazových bodech. Jinak by se zobrazilo minimální okno , resp. , protože jeho obsah je prázdný.
4. Teprve až je spuštěna metoda `window.read()`, okno se zobrazí a program čeká. Program se ukončí, když uživatel klepne na uzávěr okna.

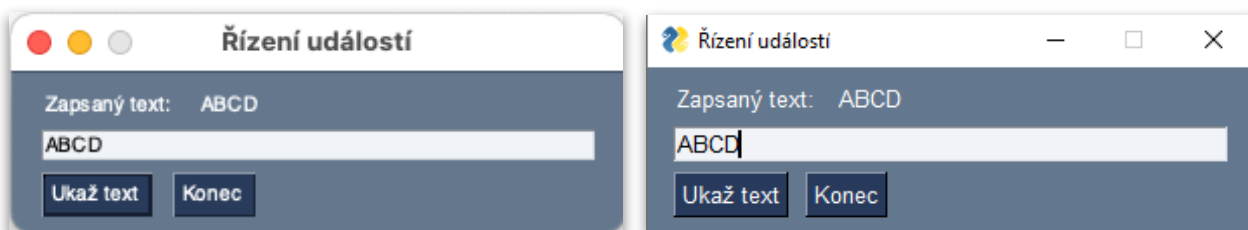
Poznámka: V balíku PySimpleGUI je základní vzhled nazývaný "theme" tmavě modrý. Dá se ale změnit jak na některý z mnoha dalších (pomocí funkce `sg.theme()`), tak na vzhled běžný v operačním systému. Jestliže vyvoláme funkci `sg.theme('Default 1')`, zobrazí se okno v podobě podle operačního systému (macOS, Windows):



Řízení událostí

V *PySimpleGui* řídicí cyklus událostí (event loop) není ukryt v základní třídě, jako v soustavě *tkinter* nebo v jiných GUI soustavách, ale je k dispozici uživateli.

Řídicí cyklus ukážeme na příkladu programu `řízení_udalosti_sg.py`, který zobrazí okno s nápisem, vstupním polem a dvěma tlačítky. Když uživatel zapíše text do vstupního pole a stiskne tlačítko "Ukaž text", zobrazí se tento text za nápisem. Když uživatel stiskne tlačítko "Konec" nebo klepne na uzávěr okna, okno zmizí a program skončí.



Konstrukce okna s klíči

V první části programu se vytvoří okno a jeho struktura: nejprve objekt rozvrhu `rozvrh` a pak objekt okna `okno` s tímto rozvrhem.

```
import PySimpleGUI as sg

rozvrh = [
    [sg.Text('Zapsaný text:'), sg.Text(size=(20,1), key='-VYSTUP-')],
    [sg.Input(key='-VSTUP-')],
    [sg.Button('Ukaž text'), sg.Button('Konec')]
]

okno = sg.Window('Řízení událostí', layout=rozvrh)
```

- 1 Rozvrh prvků v okně (layout) se zapisuje jako *seznam seznamů*, tj. seznam řádků (jako vnější seznam) a v řádku *seznam prvků* (jako vnitřní seznam). Vnější hranaté závorky představují celé okno, vnitřní hranaté závorky jednotlivé řádky.
- 2 V prvním řádku rozvrhu jsou dva *výstupní texty* třídy `Text`: první s nápisem "Zapsaný text:", druhý prázdný o délce 20 *znaků* a šířce (výšce) jednoho *řádku*. Druhý text má v sobě *klíč* označený jako `-VYSTUP-`.
- 3 V druhém řádku je *vstupní textové pole* třídy `Input` s *klíčem* `-VSTUP-`.
- 4 V třetím řádku jsou dvě *tlačítka* třídy `Button`.
- 5 Okno `window` třídy `Window` má první argument `'Řízení událostí'`, který je vidět v záhlaví. Druhý argument je `layout=rozvrh`. Oba argumenty jsou povinné.

To, co jsme dosud popsali, je konstrukce tvaru okna s některými prvky obsahujícími klíče. *Klíče* jsou zde konvenčně formované texty, které slouží k identifikaci prvků při jejich zpracování v řídicím cyklu, který následuje.

Řídicí cyklus s vyhodnocením klíčů

Druhou část programu tvoří řídicí cyklus a uzávěr okna.

```
while True: # řídicí cyklus 1
    event, values = okno.read() 2
    print(event, values)
    if event == sg.WIN_CLOSED or event == 'Konec': 3
        break
    if event == 'Ukaž text': 4
        # změní prvek -VYSTUP- tak, aby získal hodnotu z prvku -VSTUP-
        okno['-VYSTUP-'].update(values['-VSTUP-']) 5
okno.close() 6
```

- 1 Cyklus se opakuje, dokud neskončí příkazem `break` 3.
- 2 První v cyklu musí být příkaz `read()`, který čeká na událost. Nastane-li událost, příkaz `read()` ukončí čekání a výpočet pokračuje. Abychom mohli zjistit, co se v okně událo, metoda `read()` vrací dvojici (2-tuple) do proměnných konvenčně nazvaných `event, values`.
- 3 Nastala-li událost `WIN_CLOSED` (klepnutí na uzávěr okna) nebo klepnutí na tlačítko "Konec", příkazem `break` vyskočí výpočet z cyklu, okno se uzavře 6, a výpočet skončí.
- 4 Nastala-li událost klepnutí na tlačítko "Ukaž text",
- 5 provede se v okně změna metodou `update()`. Změna spočívá v tom, že hodnota, na kterou ukazuje klíč `-VSTUP-`, se zkopíruje do prvku s klíčem `-VYSTUP-`. To je prvek `sg.Text(size=(20,1), key='-VYSTUP-')` z rozvrhu okna. Do něj se tedy zapíše text ze vstupního pole `sg.Input(key='-VSTUP-')`, jestliže do něj uživatel nějaký napsal.

Výpočet se pak vrátí na příkaz `read()`, který čeká na další událost, a tak pořád dokola, dokud není cyklus ukončen příkazem `break`.

Vysvětlivky

- 1) V programu se také tisknou hodnoty `event` a `values`:

```
print(event, values)
Ukaž text {'-VSTUP-': 'ABCD'}
```

Vidíme, že objekt `values` je *slovník* (dictionary) `{'-VSTUP-': 'ABCD'}`.

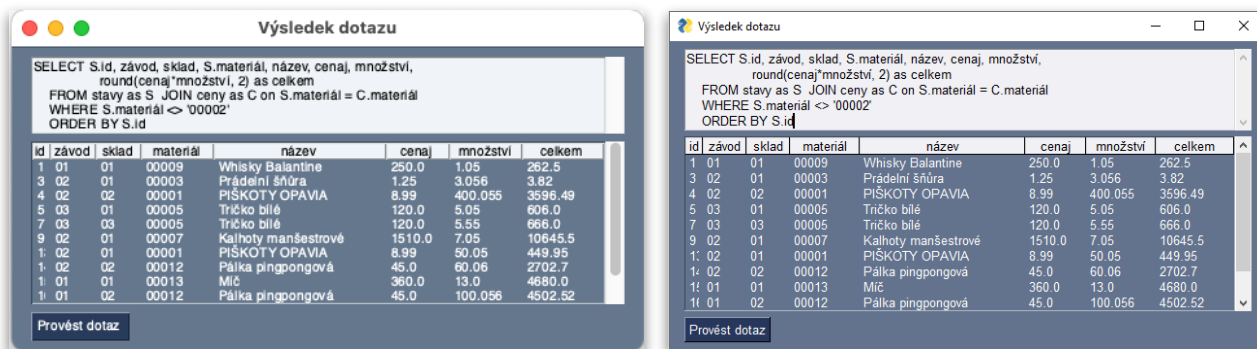
Výraz `values['-VSTUP-']` tedy znamená *čtení hodnoty* 'ABCD' ze slovníku `values` podle klíče `-VSTUP-`.

- 2) Hodnota proměnné `event` je text z definice tlačítka. Proto můžeme událost porovnávat se jménem tlačítka.
- 3) Výraz `okno['-VYSTUP-']` je zkratka pro volání metody:
`okno.find_element('-VYSTUP-')`.

Dotaz zapsaný uživatelem

Program `sqlite3_dotaz.py` na začátku zapiše do textové oblasti vzorový dotaz (příkaz `SELECT`) a jeho výsledek zobrazí v tabulce. Uživatel může text dotazu změnit nebo napsat jiný a pak stisknout tlačítko "Provést dotaz". Program pak zobrazí výsledek nového dotazu.

Platí stejné zadání jako v dřívější stejně nazvané [kapitole](#).



Výsledek dotazu

```
SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
       round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id
```

id	závod	sklad	materiál	název	cenaj	množství	celkem
1	01	01	00009	Whisky Balantine	250.0	1.05	262.5
3	02	01	00003	Prádelní šňůra	1.25	3.056	3.82
4	02	02	00001	PIŠKOTY OPAVIA	8.99	400.055	3596.49
5	03	01	00005	Tričko bílé	120.0	5.05	606.0
7	03	03	00005	Tričko bílé	120.0	5.55	666.0
9	02	01	00007	Kalhoty manšestrové	1510.0	7.05	10645.5
1	02	01	00001	PIŠKOTY OPAVIA	8.99	50.05	449.95
1	02	02	00012	Pálka pingpongová	45.0	60.06	2702.7
1	01	01	00013	Míč	360.0	13.0	4680.0
1	01	02	00012	Pálka pingpongová	45.0	100.056	4502.52

Provést dotaz

Po provedení chybného příkazu se zobrazí diagnostické okno s uvedením textu chyby. Například

```
SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
       round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.idxxx
```

zobrazí toto okno:



Hlavní část programu

Příprava a provedení dotazu poprvé

```
connection = sqlite3.connect('/Users/vzupka/PycharmProjects/PySimpleGUI/zasoby.db') # připojím databázi
connection.row_factory = sqlite3.Row # objekt s údaji o řádku tabulky
cursor = connection.cursor() # definuji objekt typu Cursor

# text vzorového SQL dotazu
text_dotazu = '''SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
                    round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id'''

cursor.execute(text_dotazu) # provedu vzorový dotaz
hlavicky, data = data_tabulky() # získám hlavičky a data pro tabulku
```

Konstrukce obrazové tabulky a okna

```
print = sg.easy_print # přesměrování tisku do ladicího okna (debug widow) 1

textova_oblast = sg.Multiline(auto_size_text=True, default_text=text_dotazu,
                               size=(50, 5), expand_x=True) 2

tabulka = sg.Table(values=data, # řádky a sloupce 3
                   headings=hlavicky, # hlavičky
                   auto_size_columns=False,
                   key='-TABLE-', # klíč tabulky
                   justification='left') # zarovnání dat v buňce

rozvrh = [[textova_oblast], [tabulka], [sg.Button("Provést dotaz")]] 4

okno = sg.Window('Výsledek dotazu', rozvrh, resizable=True) 5
```

1 Standardní funkce `print()` bude tisknout do výstupního ladicího okna místo do standardního výstupu. Výstupní okno předvolené velikosti a umístění se vytvoří při prvním provedení funkce `print()`.

2 Třída `Multiline` představuje textovou oblast, do níž může zapisovat program i uživatel a ze které může program číst. Parametr `expand_x` dovoluje rozpínání oblasti ve vodorovném směru.

3 Třída `Table` definuje obrazovou tabulku se zadaným obsahem (seznam `data`) a hlavičkami (seznam `hlavicky`), který byl předem zjištěn.

4 Rozvrh okna je dán seznamem tří řádků zapsaných jako seznamy o jednom prvku. Třída `Button` představuje tlačítko.

5 Třída `Window` definuje okno s rozvrhem `rozvrh`, má potřebnou velikost a předvolené umístění. Parametr `resizable` dovoluje roztažení okna.

Zpracování událostí v cyklu

```
while True:
    event, values = okno.read() # čtu události a hodnoty z okna 1
    if event == sg.WIN_CLOSED: # po klepnutí na uzávěr okna končím cyklus 2
        break
    if event == "Provést dotaz": 3
        try:
            cursor.execute(textova_oblast.get()) 4
            hlavicky, data = data_tabulky() # zjistím nová data z dotazu
            okno['-TABLE-'].update(values=data) 5
        except sqlite3.OperationalError as err:
            print(err) # chyba se objeví v okně pro přesměrovaný tisk

okno.close()
```

1 Jestliže v okně nastane nějaká událost, metoda `read()` zastaví výpočet a vrátí dvojici představující událost `event` a hodnoty `values`.

2 Klepnutí na uzávěr okna generuje událost v podobě čísla prezentovaného jako symbolická konstanta `WIN_CLOSED`. Nastane-li tato událost, provede se příkaz `break`, cyklus skončí a okno se uzavře metodou `close()`.

3 Klepnutí na tlačítko generuje událost v podobě textu tlačítka. V tom případě provedeme nový dotaz a zobrazíme data z výsledné tabulky.

4 Nový SQL příkaz je přečten z textové oblasti metodou `get()`.

5 Data z provedeného dotazu získáme voláním funkce `data_tabulky()` (popsané dále) a opravíme jimi obrazovou tabulku v okně metodou `update()`. Vazba mezi tabulkou a oknem je dána klíčem `-TABLE-`. Parametr `key='-TABLE-'` v definici tabulky koresponduje s argumentem v hranaté závorce u příkazu `okno['-TABLE-'].update(values=data)`.

Funkce `data_tabulky()`

```
def data_tabulky():
    hlavicky = [] # seznam hlaviček
    data = [] # řádky jako seznam seznamů (řádků - záznamů)
    zaznam = cursor.fetchone() # přečtu první záznam
    if zaznam: # aspoň jeden řádek?
        hlavicky = zaznam.keys() # dostanu seznam jmen sloupců (hlaviček)
    while zaznam: # v cyklu přidám řádek do dat a čtu další záznam
        data.append(list(zaznam)) # Row objekt musím přeměnit na seznam
        zaznam = cursor.fetchone()
    return hlavicky, data
```

Celý program

```
import PySimpleGUI as sg, sqlite3

def data_tabulky():
    hlavicky = [] # seznam hlaviček
    data = [] # řádky jako seznam seznamů (řádků - záznamů)
    zaznam = cursor.fetchone() # přečtu první záznam
    if zaznam: # aspoň jeden řádek?
        hlavicky = zaznam.keys() # dostanu seznam jmen sloupců (hlaviček)
    while zaznam: # v cyklu přidám řádek do dat a čtu další záznam
        data.append(list(zaznam)) # Row objekt musím přeměnit na seznam
        zaznam = cursor.fetchone()
    return hlavicky, data

connection = sqlite3.connect('/Users/vzupka/PycharmProjects/PySimpleGUI/zasoby.db') #
připojím databázi
connection.row_factory = sqlite3.Row # objekt ke zjišťování údajů o řádku tabulky
cursor = connection.cursor() # definuji objekt typu Cursor

# text vzorového SQL dotazu
text_dotazu = '''SELECT S.id, závod, sklad, S.materiál, název, cenaj, množství,
                    round(cenaj*množství, 2) as celkem
FROM stavy as S JOIN ceny as C on S.materiál = C.materiál
WHERE S.materiál <> '00002'
ORDER BY S.id'''

cursor.execute(text_dotazu) # provedu vzorový dotaz
hlavicky, data = data_tabulky() # získám hlavičky a data pro tabulku

print = sg.easy_print # přesměrování tisku do ladicího okna (debug widow)

# na začátku je text dotazu zapsán v této textové oblasti jako 'default_text'
textova_oblast = sg.Multiline(auto_size_text=True, default_text=text_dotazu,
                              size=(50, 6), expand_x=True)

tabulka = sg.Table(values=data, # řádky a sloupce
                  headings=hlavicky, # hlavičky
                  auto_size_columns=False,
                  key='-TABLE-', # klíč pro referenci objektů tabulky
                  justification='left') # zarovnání dat v buňce

rozvrh = [[textova_oblast], [tabulka], [sg.Button("Provést dotaz")]]

okno = sg.Window('Výsledek dotazu', rozvrh, resizable=True)

while True:
    event, values = okno.read() # čtu události a hodnoty z okna
    if event == sg.WIN_CLOSED: # po klepnutí na uzávěr okna končím cyklus
        break
    if event == "Provést dotaz": # na stisk tlačítka Provést provedu nový dotaz
        try:
            cursor.execute(textova_oblast.get()) # provést dotaz
            hlavicky, data = data_tabulky() # zjistit nová data z dotazu
            okno['-TABLE-'].update(values=data) # vložím nová data do tabulky
        except sqlite3.OperationalError as err: # testuji chybu sqlite3
            print(err) # chyba se objeví v okně pro přesměrovaný tisk

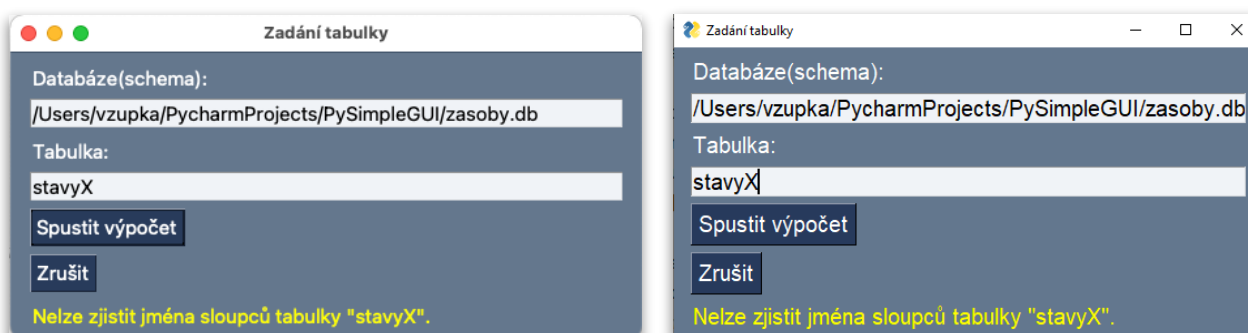
okno.close()
```

Aktualizace databázové tabulky

Program `sqlite3_update.py` provádí kompletní údržbu databázové tabulky pomocí SQL příkazů: výběru záznamů `SELECT`, přepisu záznamu `UPDATE`, vložení záznamu `INSERT` a smazání záznamu `DELETE`. Zadání je shodné jako v dřívější, stejně nazvané [kapitole](#). Provedení se v některých detailech liší. Vzhled oken bychom mohli získat stejný, kdybychom do programu přidali volání funkce `sg.theme('Default 1')` a pro macOS ještě `sg.set_options(ttk_theme='aqua')`. V textu ale ponecháme základní vzhled `sg.theme('Dark Blue 3')` pro *PySimpleGUI*.

Dialogové okno pro zadání schematu a tabulky

Hlavní část programu začíná globálními definicemi a konstrukcí dialogového okna pro zadání schematu a tabulky. (Ještě před hlavní částí jsou umístěny definice funkcí [zobrazeni_tabulky\(\)](#) a [vnitrni_cyklus\(\)](#).)



```
sg.theme('Dark Blue 3') # standard - jako když není zadáno

schema = '/Users/vzupka/PycharmProjects/PySimpleGUI/zasoby.db'
tabulka = 'stavy' # předvolená tabulka

rozvrh_zadani = [
    [sg.T('Databáze(schema):'), [sg.I(schema, k='-SCHEMA-')],
    [sg.T('Tabulka:'), [sg.I('stavy', k='-TABULKA-')],
    [sg.B('Spustit výpočet'), [sg.B('Zrušit')],
    [sg.T(text='', text_color='Yellow', key='-CHYBA-')]
]
dialog_zadani = sg.Window('Zadání tabulky', rozvrh_zadani, resizable=True,
                           font=('Default', 15), modal=True)
```

1 V rozvrhu okna jsou použity zkratky jednotlivých prvků: T místo Text, I místo Input, B místo Button, dokonce i zkratky jmen argumentů: klíč k místo key. V dokumentaci jsou zkratky uvedeny u každého prvku. Vstupní pole jsou označena klíči `-SCHEMA-` a `-TABULKA-` pro následující čtení v řídicím cyklu.

2 Výstupní text slouží k oznámení chyby žlutou barvou.

3 Okno pro dialog je označeno jako *modální*, což znamená, že uživatel může pracovat jen s tímto oknem, dokud ho neuzavře. V tomto případě jde o první a zatím jediné okno, takže s jiným oknem v tomto okamžiku ani pracovat nelze.

Pokračuje zpracováním dvou událostí (events) v dialogovém okně. Je to klepnutí na tlačítko "Zrušit" a klepnutí na tlačítko "Spustit výpočet".

```
while True: # řídicí cyklus
    event, values = dialog_zadani.read()
    if event == 'Zrušit' or event == sg.WIN_CLOSED:
        print("Končíme.")
        sys.exit(0) # opustím interpret Pythonu
    # přečtu zadání, zavřu okno a skončím cyklus
    if event == "Spustit výpočet":
        schema = values['-SCHEMA-'] 1
        tabulka = values['-TABULKA-'].

        # propojení s databází
        if not os.path.exists(schema): 2
            dialog_zadani['-CHYBA-'].update(
                f"Chyba: Neexistuje databáze {schema}")
            continue
        connection = sqlite3.connect(schema) # připojím databázi (schema)
        cursor = connection.cursor() # definuji objekt typu Cursor
        text_prikazu = f'PRAGMA table_info({tabulka})' # zjistím sloupce
        sloupce_info = cursor.execute(text_prikazu).fetchall()
        jmena_sl = [col[1] for col in sloupce_info] 3
        if not jmena_sl:
            dialog_zadani['-CHYBA-'].update(
                f'Nelze zjistit jména sloupců tabulky "{tabulka}"')
            continue
        text_prikazu = f'SELECT "{", ".join(jmena_sl)} FROM {tabulka}' 4
        try: 5
            cursor.execute(text_prikazu) # provedu dotaz na celou tabulku
        except sqlite3.Error as err:
            dialog_zadani['-CHYBA-'].update(f"Chyba příkazu: {err}")
        data = cursor.fetchall() # získám seznam všech záznamů
        dialog_zadani.close() # uzavřu okno
        break # ukončím cyklus a pokračuji v dalším výpočtu
```

1 Hodnoty ve slovníku `values` čteme podle klíčů. Víme, že klíče jsou zde vstupní texty pro `schema` a `tabulku`.

2 Vstupní hodnoty pro `schema` a `tabulku` se kontrolují a dojde-li k chybě, řídicí cyklus se opakuje, aby uživatel mohl vstupní hodnoty opravit.

3 Z informační tabulky příkazu `PRAGMA` (číslované od jedničky) vytvořím seznam jmen sloupců `jmena_sl`.

4 Příkazem `SELECT` zjistím `data` pro obrazovou tabulku.

5 Pro jistotu kontroluji příkaz.

Konstrukce hlavního okna s obrazovou tabulkou

Vytvořím kostru okna s klíči `-TABLE-`, `-WHERE-`, `-ORDER BY-`, `-COLOR-`.

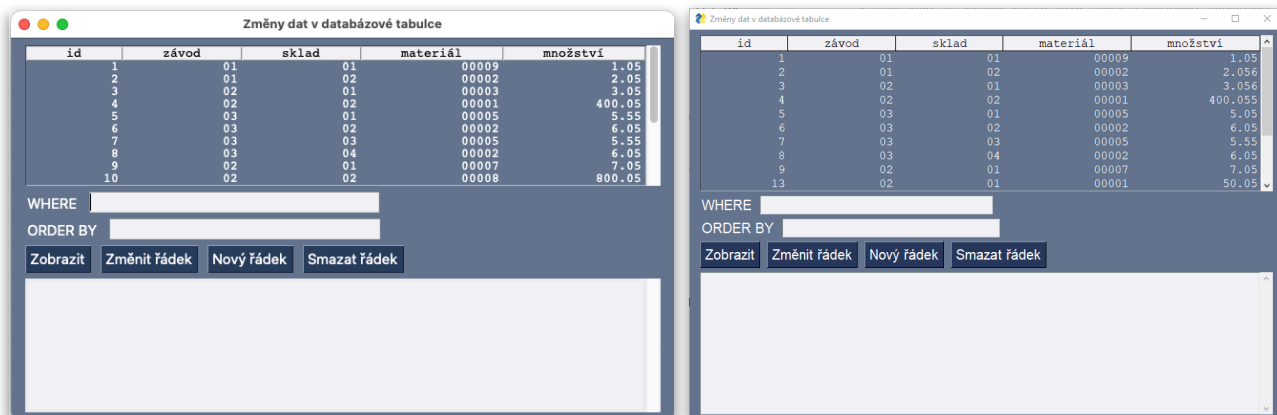
```
obraz_tabulka = sg.Table(values=data, headings=jmena_sl, key='-TABLE-',
    font=('Courier', 13), max_col_width=100,
    display_row_numbers=False, enable_events=True,
    expand_x=True, expand_y=True, enable_click_events=True,
    auto_size_columns=False) 1

rozvrh_hlavni = [
    [obraz_tabulka], [sg.T('WHERE'), sg.I(k='-WHERE-', size=(30, None))],
    [sg.T('ORDER BY'), sg.I(k='-ORDER BY-', size=(28, None))],
    [sg.B('Zobrazit'), sg.B('Změnit řádek'), sg.B('Nový řádek'),
     sg.B('Smazat řádek')]]
    # plus výstup tisku - šířka podle okna, výška 10 řádků
rozvrh_hlavni.append([sg.Multiline(size=(None, 10), reroute_stdout=True, 2
    font=('Courier', 13),
    text_color='Dark red', key='-COLOR-')])

# vytvořím hlovní okno
hlavni_okno = sg.Window('Změny dat v databázové tabulce', rozvrh_hlavni,
    resizable=True, font=('Default', 15))
```

1 Argument `auto_size_columns` umožňuje zobrazit i prázdnou tabulku (provizorium v určité verzi).

2 Do rozvrhu přidám pod tlačítka spodní okno pro zprávy, které z příkazu `print()` budou do něj přesměrovány ze standardního výstupu. Text zpráv bude mít základní barvu tmavočervenou.



Zpracování událostí v hlavním okně

Další výpočet je rozvržen do dvou cyklů – vnějšího a vnitřního. Vnější cyklus zpracovává události v hlavním okně tak, že připravuje dialogové okno s údaji pro změny. Vnitřní cyklus zpracovává události v okně změnového dialogu a je umístěn do funkce `vnitrni_cyklus()`.

Vnější cyklus

Začátek vnějšího cyklu připravuje konstrukci změnového dialogu podle toho, jaké tlačítko bylo stisknuto. Tlačítko "Zobrazit" vyvolá nové zobrazení tabulky bez ohledu na výběr řádku. Když nebyl vybrán žádný řádek, vytvoří se rozvrh dialogu pro "Nový řádek" s nulovými hodnotami sloupců. U změny a smazání řádku se hlásí chyba.

```
while True: # vnější cyklus pro hlavní okno
    event_hl, values_hl = hlavni_okno.read()
    if event_hl == sg.WIN_CLOSED:
        hlavni_okno.close() # uzavřu hlavní okno
        break # jde se až na konec programu, kde se končí

    zpracovat_zmenu = True # zatím povolím změnu řádku ❶

    if event_hl == 'Zobrazit':
        sel_rows = [] # vyprázdním seznam vybraných řádků
        zobrazeni_tabulky() # zobrazím tabulku, nebude vybrán žádný řádek ❷

    # příprava změnového okna: sestavím rozvrh okna pro příslušná tlačítka
    if (event_hl == 'Změnit řádek' or event_hl == 'Nový řádek'
        or event_hl == 'Smazat řádek'):

        size_tx = (10, 1) # velikost nápisu (znaků, řádků)
        size_in = (15, 1) # velikost vstupního pole (znaků, řádků)
        rozvrh_zmena = [] # rozvrh pro změnový dialog

        sel_rows = obraz_tabulka.SelectedRows # seznam vybraných řádků

        # stisk tlačítka 'Nový řádek' aniž byl vybrán řádek
        if event_hl == 'Nový řádek' and not sel_rows: ❸
            rozvrh_zmena = [
                # seznam pro tlačítko "Nový řádek" aniž byl vybrán řádek
                [[sg.T(jmena_sl[col_nbr], size=size_tx), sg.I('0', size=size_in)]]
                for col_nbr in range(len(jmena_sl))], # list comprehension
                [sg.B('Vložit řádek')]]

        # stisk tlačítka "Změnit řádek" nebo "Smazat řádek" aniž byl vybrán řádek
        if (event_hl == 'Změnit řádek' or event_hl == 'Smazat řádek') and not sel_rows:
            zpracovat_zmenu = False # potlačím změnu řádku, když není žádný vybrán
            print("Není vybrán řádek.")
```

❶ Příznak zpracování změny je zaveden proto, abychom mohli potlačit otevření změnového dialogu, když nebyl vybrán žádný řádek.

❷ Definice funkce `zobrazeni_tabulky()` je umístěna na začátku programu.

❸ V dialogu pro vložení zcela nového řádku (když nebyl vybrán žádný řádek) budou do vstupních polí dosazeny *znakové nuly* jako předvolené hodnoty, aby se do příkazu INSERT nedostaly prázdné hodnoty.

Pokračování vnějšího cyklu dokončí rozvrh změnového dialogu u tlačítek s vybraným řádkem `sel_row`. U tlačítek "Nový řádek" a "Smazat řádek" je část rozvrhu společná.

```
stare_hodnoty_poli = []
sel_row = 0 # číslo vybraného řádku
if sel_rows: # je-li vybrán aspoň jeden řádek
    sel_row = sel_rows[0] # беру за выбранный рядок jen první
    jm_sl = jmena_sl.copy() # vytvořím kopii seznamu sloupců 1
    for col_nbr in range(len(jmena_sl)):
        if jmena_sl[col_nbr] == 'id': # pro změnu řádku
            jm_sl.remove(jmena_sl[col_nbr]) 2
            val_id = data[sel_row][col_nbr]
        else: # přidám hodnotu ostatních sloupců do seznamu
            stare_hodnoty_poli.append(data[sel_row][col_nbr]) 3
    if event_hl == 'Změnit řádek':
        rozvrh_zmena = [
            # seznam pro tlačítko Změnit řádek
            [[sg.T(jm_sl[col_nbr], size=size_tx),
              sg.I(stare_hodnoty_poli[col_nbr], size=size_in)]]
            for col_nbr in range(len(jm_sl))], # list comprehension 4
            [sg.B('Přepsat')] ]

# sestavím společný seznam pro tlačítka Nový řádek a Smazat řádek
seznam_smazat = [[sg.T(jmena_sl[col_nbr], size=size_tx),
                   sg.I(data[sel_row][col_nbr], size=size_in)]]
                   for col_nbr in range(len(jmena_sl))] 4
if event_hl == 'Nový řádek':
    rozvrh_zmena = [
        seznam_smazat, [sg.B('Vložit řádek')] ]

if event_hl == 'Smazat řádek':
    rozvrh_zmena = [
        seznam_smazat, [sg.B('Opravdu smazat')], [sg.B('Zrušit mazání')]]
]
```

- 1 Vytvořím a upravím seznam jmen sloupců `jm_sl`.
- 2 Pro změnu řádku odstraním sloupec "id", aby se nezobrazoval a nedal se tedy přepsat. U něj si ale zapamatuji jeho hodnotu `val_id` pro použití v příkazech UPDATE a DELETE.
- 3 Vytvořím seznam `stare_hodnoty_poli` z původních hodnot vstupních polí vybraného řádku.
- 4 Rozvrh prvků pod sebou se provádí cyklickým výpočtem podle čísla sloupce v produkčním výrazu seznamu – *list comprehension*. Je to poněkud neprůhledné, ale kratší o 2 řádky než například tato alternativa:

```
seznam = []
for col_nbr in range(len(jmena_sl)):
    seznam += [sg.T(jmena_sl[col_nbr], size=size_tx),
               sg.I(data[sel_row][col_nbr], size=size_in)]
seznam_smazat = [seznam]
```

Ve vnějším cyklu se konečně vytvoří okno změnového dialogu podle tlačítka, které je vyvolalo:

```
dialog_zmena = sg.Window('Změna dat', rozvrh_zmena, resizable=True,  
                          font=('Default', 15), modal=True)
```

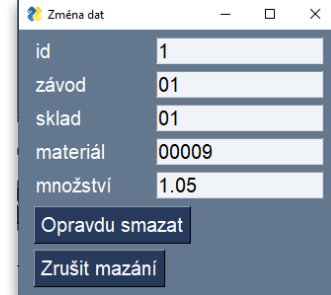
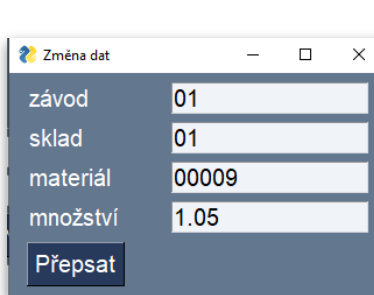
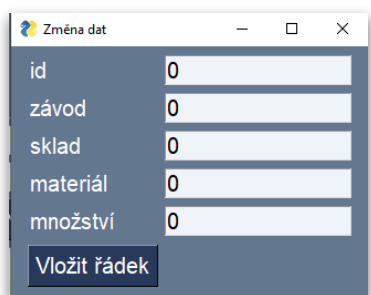
Nový řádek



Změnit řádek



Smazat řádek



Na konci vnějšího cyklu hlavního okna se spustí [vnitřní cyklus](#) pro zpracování událostí ve změnovém dialogu, čímž hlavní část programu končí.

```
vnitrni_cyklus()
```

Vnitřní cyklus

Vnitřní cyklus je umístěn do funkce a ta je pochopitelně umístěna u začátku programu.

```
def vnitřni_cyklus():

    db = sqlite_utils.Database(connection) # přístup k údajům o databázi
    # získám slovník typů sloupců tabulky ve tvaru <class 'str'>, <class 'int'> atd.
    typy_sloupcu = db[tabelka].columns_dict

    while True: # vnitřní cyklus: zpracování změnového okna
        if not zpracovat_zmenu: break # když není co zpracovat, končím cyklus 1
        # změnové okno se zpracuje jen když je vybrán řádek
        # nebo když se vkládá zcela nový řádek s nulovými hodnotami
        event_zm, values_zm = dialog_zmena.read()
        if event_zm == sg.WIN_CLOSED:
            dialog_zmena.close()
            break
        # -----

        if event_zm == 'Přepsat':
            sloupce = [] # pro UPDATE SET = (sloupce) = ( hodnoty )
            hodnoty = [] # pro UPDATE SET = (sloupce) = ( hodnoty )
            # sestavím seznam jmen sloupců a hodnot přečtených z textových polí
            for col_nbr in range(len(values_zm)):
                # hodnoty se musí porovnávat textově, i když jsou číselné
                if str(stare_hodnoty_poli[col_nbr]) != str(values_zm[col_nbr]):
                    sloupce.append(jm_sl[col_nbr]) 2
                    if repr(typy_sloupcu[jm_sl[col_nbr]]) == "<class 'str'>": 3
                        hodnoty.append(f"'{values_zm[col_nbr]}'") # textově s apostrofy
                    else:
                        hodnoty.append(f"{values_zm[col_nbr]}") # textově bez apostrofů
            if sloupce: # není-li seznam sloupců prázdný
                text_prikazu = f"UPDATE {tabelka}
SET ({', '.join(sloupce)}) = ({', '.join(hodnoty)})
WHERE id = {val_id}" # přepíšu sloupce kromě 'id' 4
                hlavni_okno['-COLOR-'].update(text_color='Black') 5
                print(text_prikazu)
                try:
                    cursor.execute(text_prikazu)
                    connection.commit()
                    zobrazeni_tabulky() # zobrazím změněnou tabulku znovu
                except sqlite3.Error as err:
                    hlavni_okno['-COLOR-'].update(text_color='Dark red') 5
                    print(f"Chyba: {err}")
            else:
                print('Záznam se nemění.')
            dialog_zmena.close()
```

1 [Příznak zpracování změny](#) byl zaveden proto, abych mohl potlačit otevření změnového dialogu, když nebyl vybrán žádný řádek a bylo stisknuto jiné tlačítko než "Nový řádek".

2 Seznam jmen sloupců jm_sl byl vytvořen ve [vnějším cyklu](#).

3 Test na typ sloupce, zde na typ str (viz modul sqlite_utils)

4 Hodnota val_id sloupce "id" byla také vytvořena ve [vnějším cyklu](#).

5 Určuje se barva výstupního textu, který se má tisknout. Klíč -COLOR- je definován v definici výstupního okna třídy Multiline.

Zpracování ostatních tlačítek je již jednodušší. Po každé provedené akci se dialogové okno zavře.

```
# -----
if event_zm == 'Vložit řádek':
    hodnoty = []
    # hodnoty pro INSERT INTO tabulka (sloupce) VALUES (hodnoty)
    for col_nbr in range(len(values_zm)):
        if repr(typy_sloupcu[jmena_sl[col_nbr]]) == "<class 'str'>":
            hodnoty.append(f"'{values_zm[col_nbr]}'") # textově s apostrofy
        else:
            hodnoty.append(f"{values_zm[col_nbr]}") # textově bez apostrofů
    text_prikazu = f"INSERT INTO {tabulka} ({', '.join(jmena_sl)}) VALUES ({', '.join(hodnoty)})"
    hlavni_okno['-COLOR-'].update(text_color='Black')
    print(text_prikazu)
    try:
        cursor.execute(text_prikazu)
        connection.commit()
        zobrazeni_tabulky() # zobrazím SQL tabulku v obrazové tabulce znovu
    except sqlite3.Error as err:
        hlavni_okno['-COLOR-'].update(text_color='Dark red')
        print(f"Chyba: {err}")
    dialog_zmena.close()

# -----

if event_zm == 'Opravdu smazat':
    text_prikazu = f"DELETE FROM {tabulka} WHERE id = {val_id}"
    hlavni_okno['-COLOR-'].update(text_color='Black')
    print(text_prikazu)
    try:
        cursor.execute(text_prikazu)
        connection.commit()
        zobrazeni_tabulky() # zobrazím SQL tabulku v obrazové tabulce znovu
    except sqlite3.Error as err:
        hlavni_okno['-COLOR-'].update(text_color='Dark red')
        print(f"Chyba: {err}")
    dialog_zmena.close()

# -----

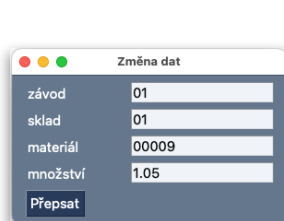
if event_zm == 'Zrušit mazání':
    dialog_zmena.close()

# -----
```

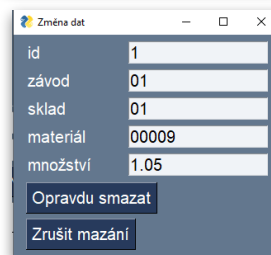
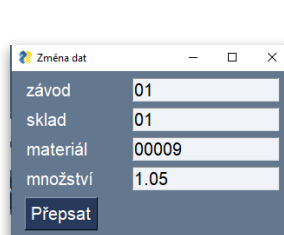
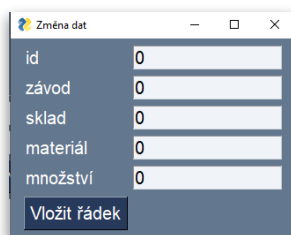
Nový řádek



Změnit řádek



Smazat řádek



Zobrazení tabulky

Funkce *zobrazení databázové tabulky* v obrazové tabulce je poněkud jednodušší než v balíku *tkinter*. Funkce je volána vícekrát z vnějšího i vnitřního cyklu.

```
def zobrazeni_tabulky():
    global data
    where = values_hl['-WHERE-']
    orderby = values_hl['-ORDER BY-']
    text_prikazu = f'''SELECT {"", ".join(jmena_sl)} FROM {tabulka}'''
    if where:
        text_prikazu += f'''
WHERE {where} '''
    if orderby:
        text_prikazu += f'''
ORDER BY {orderby} '''
    text_prikazu += ''' LIMIT 1000 '''
    hlavni_okno['-COLOR-'].update(text_color='Black')
    try:
        cursor.execute(text_prikazu) # provedu dotaz
        data = cursor.fetchall() # získám seznam všech záznamů
        # vložím nová data do tabulky, hlavičky už jsou v tabulce
        hlavni_okno['-TABLE-'].update(values=data) 1
    except sqlite3.Error as err:
        hlavni_okno['-COLOR-'].update(text_color='Dark red')
        print(f"Chyba příkazu: {err}")
```

1 Před zápisem nových dat do obrazové tabulky není nutné dosavadní data odstraňovat, protože metoda `update()` to zařídí sama.

Celý program

```
import PySimpleGUI as sg, sqlite3, sys, os
import sqlite_utils

# funkce zobrazení databázové tabulky v obrazové tabulce
# -----
def zobrazeni_tabulky():
    global data, color
    where = values_hl['-WHERE-']
    orderby = values_hl['-ORDER BY-']
    text_prikazu = f'''SELECT {", ".join(jmena_sl)} FROM {tabulka}'''
    if where:
        text_prikazu += f'''
WHERE {where} '''
    if orderby:
        text_prikazu += f'''
ORDER BY {orderby} '''
    text_prikazu += f''' LIMIT 1000 '''
    hlavni_okno['-COLOR-'].update(text_color='Black')
    try:
        cursor.execute(text_prikazu) # provedu dotaz
        data = cursor.fetchall() # získám seznam všech záznamů
        # vložím nová data do tabulky, hlavičky už jsou v tabulce
        hlavni_okno['-TABLE-'].update(values=data)
    except sqlite3.Error as err:
        hlavni_okno['-COLOR-'].update(text_color='Dark red')
        print(f"Chyba příkazu: {err}")
# -----

# funkce zpracování vnitřního cyklu pro změnový dialog
# -----
def vnitřni_cyklus():
    global color

    db = sqlite_utils.Database(connection) # přístup k údajům o databázi
    # získám slovník typů sloupců tabulky ve tvaru <class 'str'>, <class 'int'> atd.
    typy_sloupcu = db[tableka].columns_dict

    while True: # vnitřní cyklus: zpracování změnového okna
        if not zpracovat_zmenu: break # když není co zpracovat, končím cyklus
        # změnové okno se zpracuje jen když je vybrán řádek
        # nebo když se vkládá zcela nový řádek s nulovými hodnotami
        event_zm, values_zm = dialog_zmena.read()
        if event_zm == sg.WIN_CLOSED:
            dialog_zmena.close()
            break

    # -----

    if event_zm == 'Přepsat':
        sloupce = [] # pro UPDATE SET = (sloupce) = (hodnoty)
        hodnoty = [] # pro UPDATE SET = (sloupce) = (hodnoty)
        # sestavím seznam jmen sloupců a hodnot přečtených z textových polí
        for col_nbr in range(len(values_zm)):
            if (stare_hodnoty_poli[col_nbr]) != values_zm[col_nbr]:
                sloupce.append(jm_sl[col_nbr])
                if repr(typy_sloupcu[jm_sl[col_nbr]]) == "<class 'str'>":
                    hodnoty.append(f'''{values_zm[col_nbr]}''') # textově s apostrofy
                else:
                    hodnoty.append(f'''{values_zm[col_nbr]}''') # textově bez apostrofů
        if sloupce: # není-li seznam sloupců prázdný
            text_prikazu = f'''UPDATE {tabulka}
SET ({', '.join(sloupce)}) = ({', '.join(hodnoty)})'''
```

```

WHERE id = {val_id}""" # přepíšu sloupce kromě 'id'
hlavni_okno['-COLOR-'].update(text_color='Black')
print(text_prikazu)
try:
    cursor.execute(text_prikazu)
    connection.commit()
    zobrazeni_tabulky() # zobrazím změněnou tabulku znovu
except sqlite3.Error as err:
    hlavni_okno['-COLOR-'].update(text_color='Dark red')
    print(f"Chyba: {err}")
else:
    print('Záznam se nemění.')
dialog_zmena.close()

# -----

if event_zm == 'Vložit řádek':
    hodnoty = []
    # hodnoty pro INSERT INTO tabulka (sloupce) VALUES (hodnoty)
    for col_nbr in range(len(values_zm)):
        if repr(typy_sloupcu[jmena_sl[col_nbr]]) == "<class 'str'>":
            hodnoty.append(f"'{values_zm[col_nbr]}'") # textově s apostrofy
        else:
            hodnoty.append(f"{values_zm[col_nbr]}") # textově bez apostrofů
    text_prikazu = f"""INSERT INTO {tabulka}
({', '.join(jmena_sl)} VALUES ({', '.join(hodnoty)})"""
    hlavni_okno['-COLOR-'].update(text_color='Black')
    print(text_prikazu)
    try:
        cursor.execute(text_prikazu)
        connection.commit()
        zobrazeni_tabulky() # zobrazím SQL tabulku v obrazové tabulce znovu
    except sqlite3.Error as err:
        hlavni_okno['-COLOR-'].update(text_color='Dark red')
        print(f"Chyba: {err}")
    dialog_zmena.close()

# -----

if event_zm == 'Opravdu smazat':
    text_prikazu = f"DELETE FROM {tabulka} WHERE id = {val_id}"
    hlavni_okno['-COLOR-'].update(text_color='Black')
    print(text_prikazu)
    try:
        cursor.execute(text_prikazu)
        connection.commit()
        zobrazeni_tabulky() # zobrazím SQL tabulku v obrazové tabulce znovu
    except sqlite3.Error as err:
        hlavni_okno['-COLOR-'].update(text_color='Dark red')
        print(f"Chyba: {err}")
    dialog_zmena.close()

# -----

if event_zm == 'Zrušit mazání':
    dialog_zmena.close()

# -----

#sg.theme('Default 1') # barevnost je převzata z operačního systému
#sg.theme('Gray Gray Gray') # barevnost je samá šedá
#sg.theme('Light Green 6')
sg.theme('Dark Blue 3') # standard - jako když není zadáno

# 'default', 'clam', 'alt', 'classic', jen pro macOS: 'aqua', jen pro Windows:
# 'winnative', 'vista', 'xpnative'
#sg.set_options(ttk_theme='aqua') # tvar tlačítek jako v macOS
#sg.set_options(ttk_theme='alt') # standard - jako když není zadáno

```

```

# globální proměnné
schema = '/Users/vzupka/PycharmProjects/PySimpleGUI/zasoby.db' # předvolené schema
tabulka = 'stavy' # předvolená tabulka
# -----

# okno pro zadání databáze a tabulky
# =====
# rozvrh okna pro zadání databáze a tabulky
rozvrh_zadani = [
    [sg.T('Databáze(schema):')], [sg.I(schema, k='-SCHEMA-')],
    [sg.T('Tabulka:')], [sg.I(tabulka, k='-TABULKA-')],
    [sg.B('Spustit výpočet')], [sg.B('Zrušit')],
    [sg.T(text='', text_color='Yellow', key='-CHYBA-')]
]

# plus výstup tisku - šířka podle okna, výška 10 řádků
#rozvrh_zadani.append([sg.Output(size=(60, 10), font=('Courier', 13))])

dialog_zadani = sg.Window('Zadání tabulky', rozvrh_zadani, resizable=True,
                           font=('Default', 15), modal=True)
# -----

# zpracování událostí v dialogovém okně pro zadání databáze a tabulky
while True: # řídicí cyklus
    event, values = dialog_zadani.read()
    if event == 'Zrušit' or event == sg.WIN_CLOSED:
        print("Končíme.")
        sys.exit(0) # opustím interpret Pythonu
    # přečtu zadání, zavřu okno a skončím cyklus
    if event == "Spustit výpočet":
        schema = values['-SCHEMA-']
        tabulka = values['-TABULKA-']
        # propojení s databází
        # -----
        if not os.path.exists(schema):
            dialog_zadani['-CHYBA-'].update(
                f"Chyba: Neexistuje databáze {schema}")
            #print(f"Chyba: Neexistuje databáze {schema}")
            continue
        connection = sqlite3.connect(schema) # připojím databázi (schema)
        cursor = connection.cursor() # definuji objekt typu Cursor
        text_prikazu = f'PRAGMA table_info({tabulka})' # zjistím sloupce z "table_info"
        sloupce_info = cursor.execute(text_prikazu).fetchall()
        jmena_sl = [col[1] for col in sloupce_info] # seznam jmen sloupců
        if not jmena_sl:
            dialog_zadani['-CHYBA-'].update(
                f'Nelze zjistit jména sloupců tabulky "{tabulka}"')
            continue
        text_prikazu = f'SELECT {", ".join(jmena_sl)} FROM {tabulka}'
        try:
            cursor.execute(text_prikazu) # provedu dotaz na celou tabulku
        except sqlite3.Error as err:
            dialog_zadani['-CHYBA-'].update(f"Chyba příkazu: {err}")
            continue
        data = cursor.fetchall() # získám seznam všech záznamů
        dialog_zadani.close() # uzavřu okno
        break # ukončím cyklus a pokračuji v dalším výpočtu

    # -----
# -----

# hlavní okno s tabulkou dat
# =====
# vytvoření hlavního okna a tabulky s daty
# vytvořím kostru obrazové tabulky s klíči -TABLE-, -WHERE-, -ORDER BY-

```

```

obraz_tabulka = sg.Table(values=data, headings=jmena_sl, key='-TABLE-',
    font=('Courier', 13), max_col_width=100,
    display_row_numbers=False, enable_events=True,
    expand_x=True, expand_y=True, enable_click_events=True,
    auto_size_columns=False) # pomůže, když je prázdná tabulka

rozvrh_hlavni = [
    [obraz_tabulka], [sg.T('WHERE'), sg.I(k='-WHERE-', size=(30, None))],
    [sg.T('ORDER BY'), sg.I(k='-ORDER BY-', size=(28, None))],
    [sg.B('Zobrazit'), sg.B('Změnit řádek'), sg.B('Nový řádek'),
     sg.B('Smazat řádek')]]
    # plus výstup tisku - šířka podle okna, výška 10 řádků
rozvrh_hlavni.append([sg.Multiline(size=(None, 10), reroute_stdout=True,
    font=('Courier', 13),
    text_color='Dark red', key='-COLOR-')])

    # vytvořím hlavní okno
hlavni_okno = sg.Window('Změny dat v databázové tabulce', rozvrh_hlavni,
    resizable=True, font=('Default', 15))
# -----

# Další výpočet je rozvržen do dvou cyklů – vnějšího a vnitřního.
# Vnější cyklus zpracovává události v hlavním okně
# a vnitřní cyklus zpracovává události v okně změnového dialogu.

# vnější cyklus: zpracování hlavního okna
while True:
    event_hl, values_hl = hlavni_okno.read()
    #print(event_hl, values_hl)
    if event_hl == sg.WIN_CLOSED:
        hlavni_okno.close() # uzavřu hlavní okno
        break # jde se až na konec programu, kde se končí
    zpracovat_zmenu = True # zatím povolím změnu řádku
    # -----
    # nové zobrazení tabulky
    if event_hl == 'Zobrazit':
        sel_rows = [] # vyprázdním seznam vybraných řádků
        zobrazeni_tabulky() # zobrazím tabulku, nebude vybrán žádný řádek
    # -----

    # příprava změnového okna: sestavím rozvrh okna pro příslušná tlačítka
    if (event_hl == 'Změnit řádek' or event_hl == 'Nový řádek'
        or event_hl == 'Smazat řádek'):

        size_tx = (10, 1) # velikost nápisu (znaků, řádků)
        size_in = (15, 1) # velikost vstupního pole (znaků, řádků)
        rozvrh_zmena = [] # rozvrh pro změnový dialog

        sel_rows = obraz_tabulka.SelectedRows # seznam vybraných řádků

        # stisk tlačítka 'Nový řádek' aniž byl vybrán řádek
        if event_hl == 'Nový řádek' and not sel_rows:
            rozvrh_zmena = [
                # seznam pro tlačítko Nový řádek, když nebyl žádný řádek vybrán
                # dosadím samé znakové nuly
                [[sg.T(jmena_sl[col_nbr], size=size_tx), sg.I('0', size=size_in)]]
                for col_nbr in range(len(jmena_sl))], # produkční výraz
                [sg.B('Vložit řádek')]]

        # stisk tlačítka "Změnit řádek" nebo "Smazat řádek" aniž byl vybrán řádek
        if (event_hl == 'Změnit řádek' or event_hl == 'Smazat řádek') and not sel_rows:
            zpracovat_zmenu = False # potlačím změnu řádku, když není žádný vybrán
            print("Není vybrán řádek.")

        # Upravím seznam jmen sloupců podle druhu změny a vytvořím seznam původních
        # hodnot polí. U změny řádku vynechám sloupec "id".

```

```

stare_hodnoty_poli = []
sel_row = 0 # číslo vybraného řádku
if sel_rows: # je-li vybrán aspoň jeden řádek
    sel_row = sel_rows[0] # беру за выбранный рядок jen první
    jm_sl = jmena_sl.copy() # vytvořím kopii seznamu sloupců pro změnu řádku
    for col_nbr in range(len(jmena_sl)):
        if jmena_sl[col_nbr] == 'id': # pro změnu řádku
            jm_sl.remove(jmena_sl[col_nbr]) # vynechám sloupec 'id'
            val_id = data[sel_row][col_nbr] # a přečtu hodnotu ve sloupci 'id'
        else: # přidám hodnotu ostatních sloupců do seznamu
            stare_hodnoty_poli.append(data[sel_row][col_nbr])

if event_hl == 'Změnit řádek':
    rozvrh_zmena = [
        # seznam pro tlačítka Změnit řádek
        [[sg.T(jm_sl[col_nbr], size=size_tx),
          sg.I(stare_hodnoty_poli[col_nbr], size=size_in)]]
        for col_nbr in range(len(jm_sl))], # list comprehension
        [sg.B('Přepsat')] ]

# sestavím společný seznam pro tlačítka Nový řádek a Smazat řádek
seznam_smazat = [[sg.T(jmena_sl[col_nbr], size=size_tx),
                   sg.I(data[sel_row][col_nbr], size=size_in)]]
                   for col_nbr in range(len(jmena_sl))]

if event_hl == 'Nový řádek':
    rozvrh_zmena = [
        seznam_smazat, [sg.B('Vložit řádek')] ]

if event_hl == 'Smazat řádek':
    rozvrh_zmena = [
        seznam_smazat, [sg.B('Opravdu smazat')], [sg.B('Zrušit mazání')] ]

dialog_zmena = sg.Window('Změna dat', rozvrh_zmena, resizable=True,
                          font=('Default', 15), modal=True)

# -----
hlavni_okno.disappear() # bude neviditelné
vnitrni_cyklus() # funkce zpracuje změny
hlavni_okno.reappear() # znovu bude viditelné
# -----

```

Distribuce programu jako aplikace

Následující postupy fungují u programů z balíků *wxPython*, *PySimpleGUI*, *Tkinter*.

Pro Windows a macOS

Postup ukážeme pro určitost na příkladu programu **tkdb_update.py** – SQL dotaz zapsaný uživatelem.

1. Vytvoření aplikace provedeme v projektu prostředí *PyCharm* použitím *Terminálu*. Potřebujeme k tomu modul *PyInstaller*. Jeho popis je na stránce <https://www.pyinstaller.org>. Modul instalujeme příkazem

```
pip3.9 install pyinstaller
```

2. V prostředí *macOS* k tomu potřebujeme ještě modul *py2app*, který instalujeme příkazem

```
pip3.9 install py2app
```

3. Spustíme instalátor *PyInstaller*.

```
pyinstaller -F --noconsole tkdb_update.py
```

Tímto spuštěním vzniknou (nebo se doplní) adresáře `build` (vytvoření) a `dist` (distribution).

Windows

Do adresáře `dist` bude umístěna aplikace **tkdb_update.exe**. Spustíme ji poklepnáním na ikonu



Aplikace obsahuje vše, co je třeba k jejímu provozu, včetně kompletní verze Pythonu. Odtud lze aplikaci kopírovat kamkoliv jinam a vytvářet z ní zástupce.

macOS

Do adresáře `dist` bude umístěna aplikace **tkdb_update.app**. Tato aplikace je však nespolehlivá, nedoporučuje se k použití.

Kromě ikonové aplikace bude v adresáři `dist` ještě spustitelný soubor Unix **tkdb_update**, který je spolehlivý. Poklepnáním se spustí v *Terminálu*.

Jen pro macOS

Postup ukážeme pro určitost na příkladu s programem **sqlite3_update.py** aktualizace databázové tabulky z prostředí *PySimpleGUI*.

1. Vytvoření aplikace provedeme v projektu v *PyCharm* použitím *Terminálu*. Potřebujeme k tomu modul **py2app**, který instalujeme příkazem

```
pip3.9 install py2app
```

2. Pak ještě potřebujeme skript **setup.py**, který je potřeba k vytvoření aplikace. K jeho vytvoření použijeme skript **py2applet**:

```
py2applet --make-setup sqlite3_update.py
```

Existuje-li již skript **setup.py**, ukáže se dotaz

```
Existing setup.py detected, replace? [Y/n]
```

Po odpovědi **y** se existující skript přepíše novým a ukáže se odpověď

```
Wrote setup.py
```

Skript **setup.py** vznikne v běžném adresáři.

3. Nakonec vytvoříme aplikaci pomocí skriptu **setup.py** a modulu **py2app**. Spuštěním vzniknou (nebo se doplní) adresáře **build** (vytvoření) a **dist** (distribution).

```
python3.9 setup.py py2app
```

Aplikace **sqlite3_update.app** bude umístěna do adresáře **dist**. Spustíme ji poklepáním na ikonu



Aplikace obsahuje vše, co je třeba k jejímu provozu, včetně kompletní verze Pythonu. Odtud lze aplikaci kopírovat kamkoliv jinam a vytvářet z ní zástupce.